

Appunti Explainable Artificial Intelligence

Francesco Acciaoli
Mattia Filipponi
Elia Matteini

A.A. 2025/2026

Indice

1. Concetti generali	1
1.1. Perché abbiamo bisogno della XAI?	1
1.2. The alignment problem	1
1.2.1. Disuguaglianze e BIAS sociali	2
1.2.2. Correlazioni spurie in ambito medico (Artefact Bias)	2
1.2.3. Vulnerabilità dei modelli "Scatola Nera" (Adversarial attacks)	2
1.2.4. L'IA che prende "scorciatoie" (Bias di Rappresentazione e nei Dati)	2
1.3. AI Act	3
1.4. Interpretabilità vs Explainability	3
1.5. Interpretabilità vs Accuracy	3
1.5.1. Tassonomia degli approcci	4
2. Bias e Fairness	4
2.1. Bias nei dati	4
2.2. Paradosso di Simpson	5
2.3. Trattamento dei calcoli renali	5
2.4. Mortalità per Covid-19	6
2.5. Bias negli algoritmi	6
3. Fairness	6
3.1. Come imporre la Fairness	7
4. Valutare l'explainability	7
4.1. Application-grounded evaluation	8
4.2. Human-grounded evaluation	8
4.3. Functionally-grounded evaluation	8
4.4. Metriche di valutazione	8
4.4.1. Quantitative performance metrics (col contributo umano)	8
4.4.2. Auxiliary or proxy performance metrics	8
4.5. Approfondimenti sulle metriche	9
4.5.1. Plausibility (Similarity?)	9
4.5.2. Fidelity	9
4.5.3. Model parameter randomization check	9
4.5.4. Incremental feature deletion	9
5. Data visualization	10
5.1. Utilizzo dei grafici	10
6. Interpretability by design	10
6.1. Decision Tree	10
6.2. Decision Rules	12
6.3. Sequential covering	13
6.4. Bayesian Rule Lists	14
6.5. Modelli Lineari	16
6.5.1. Rule-fit Algorithm	16
6.6. Optimal Decision Tree	17
6.6.1. Ingredienti dell'algoritmo	17
6.7. Gosdt algorithm	19
6.7.1. Le condizioni 1 e 2	19
6.7.2. Come si aggiornano il lower e upper bound *	19

6.7.2.1.	Intuizione per A	20
6.7.2.2.	Intuizione per B	20
6.8.	Generalized Additive Models (GAM)	20
6.9.	Explainable Boosting Machines	21
6.9.1.	Meccanismo di training	22
6.9.2.	Neural Additive models (cenni)	23
6.10.	Prototype-based Neural Network	24
6.11.	ILP - Programmazione logica induttiva	26
6.11.1.	Semantica dell'ILP	27
6.11.2.	Metodologie di Learning	28
6.11.3.	Caratteristiche di un sistema di ILP	28
6.12.	4. Il Sistema Aleph	28
6.13.	5. Il problema dei treni di Michalski	29
7.	Approcci neurosimbolici	30
7.1.	Simbolico vs Concessionista	30
7.2.	Approcci pioneristici: KBANNs	30
7.3.	NeSy e SRL/StarAI	31
7.3.1.	SRL/StarAI	31
7.3.2.	Nesy	31
7.4.	Classic SRL/StarAI tasks (problems)	32
7.4.1.	Inference	32
7.4.1.1.	Logical inference	33
7.4.1.2.	Probabilistic inference	33
7.4.2.	Parameter Learning	33
7.4.3.	Structure Learning	34
7.4.4.	Collective Classification	34
7.5.	Modelli e framework	34
7.5.1.	Markov Logic Networks (MLN)	34
7.5.1.1.	Inferenza	35
7.5.2.	ProbLog	35
7.5.3.	MLN vs ProbLog	35
8.	Spiegazioni Post-Hoc (Spiegazioni a posteriori)	36
8.1.	Model Explanation	37
8.2.	Outcome Explanation	37
8.3.	LIME - Local Interpretable Model-agnostic Explanations	38
8.4.	Ingredienti di LIME	39
8.5.	Lime Algorithm (Sparse Linear Explanations)	39
8.6.	PredDiff (Prediction difference)	40
8.6.1.	Spiegazione tramite rimozione	40
8.6.2.	Formulazione matematica	40
8.6.3.	Come si rimuove la feature?	41
8.6.4.	Limiti e criticità	41
8.7.	Game theory	42
8.7.1.	Il Dilemma del Prigioniero e l'Equilibrio di Nash	42
8.7.2.	I valori di Shapley per le coalizioni	42
8.8.	SHAP (SHapley Additive exPlanations)	43

8.8.1.	Shapley values	43
8.8.2.	Applicazione di SHAP a AI/ML	44
8.8.3.	Problema computazionale	45
8.8.3.1.	Tecniche per approssimare gli Shapley values	45
8.8.4.	Come possiamo ottenere statistiche globali a partire dalle spiegazioni locali date dalle Shapley values?	45
8.9.	KernelSHAP ($\approx$ LIME)	46
8.9.1.	Schema dell'algoritmo	46
8.9.2.	LIME e KernelSHAP	47
8.9.2.1.	Proprietà di attribuzione delle feature	47
8.10.	Metodi basati sul gradiente e Mappe di salienza	48
8.10.1.	Mappe di salienza	48
8.10.1.1.	Smooth-grad	48
8.10.1.2.	Grad-cam	48
8.10.1.2.1.	Algoritmo	49
8.10.1.3.	Guided Gradcam	49
8.10.2.	Sanity checks per Mappe di salienza	49
8.11.	Modelli con Attention	49
8.12.	Ceteris Paribus Plots	52
8.12.1.	Individual conditional expectation (ICE)	53
8.12.2.	Partial dependence plots	53
8.13.	Permutation feature importance	53

1. Concetti generali

L'intelligenza artificiale spiegabile (XAI) esplora e studia metodi per produrre o integrare modelli di IA al fine di rendere accessibile e interpretabile la loro logica interna e i loro risultati, rendendo questo processo comprensibile agli esseri umani.

Il problema principale che la XAI affronta è che molti modelli odierni, come le reti neurali profonde, operano come delle "scatole nere" (black-box). Di fronte a una decisione o raccomandazione presa dall'IA, gli utenti sono spesso confusi e si pongono domande a cui il sistema non sa rispondere esplicitamente, come: *"Perché l'hai fatto?"*, *"Perché non hai scelto un'alternativa?"*, *"Quando hai successo o fallisci?"* e *"Come posso correggere un errore?"*

1.1. Perché abbiamo bisogno della XAI?

Abbiamo tre ragioni principali:

1. **Per comprendere le decisioni dell'AI:** Fornendo spiegazioni, il sistema passa dall'essere una scatola nera a generare previsioni chiare e trasparenti. L'utente può così capire i veri motivi dietro una decisione.
2. **Per migliorare i modelli di AI:** La comprensione genera un ciclo di feedback tra il prodotto e gli esseri umani. Sapere esattamente in quali contesti il modello ha successo o fallisce, permette di intervenire per migliorarlo, creando in ultima analisi un rapporto di fiducia.
3. **Per costruire sistemi di AI affidabili (Trustworthy AI):** La trasparenza è uno dei pilastri essenziali per un'IA di cui ci si possa fidare, insieme ad altri requisiti come la supervisione umana, la robustezza tecnica e la sicurezza, la governance dei dati, l'equità (fairness), il benessere sociale e ambientale, e la responsabilità (accountability).

1.2. The alignment problem

Attualmente, la vera sfida dell'Intelligenza Artificiale è il cosiddetto problema dell'allineamento. Pur avendo capacità straordinarie, questi sistemi non possiedono meccanismi intrinseci che li allineino ai valori umani. Non hanno un'etica preimpostata o delle istruzioni di base che garantiscano risultati a nostro esclusivo vantaggio. I modelli linguistici (LLM), in particolare, operano basandosi unicamente sulla predizione di token, senza alcuna architettura interna volta a garantire tale allineamento con l'umanità.

Pensiamo al settore bancario: il GDPR stabilisce che l'utente ha diritto a ricevere una spiegazione chiara sulle decisioni automatizzate che lo riguardano. Tuttavia, c'è un problema intrinseco nei dataset usati per addestrare questi sistemi. Poiché i dati riflettono il passato, i modelli imparano che i mutui vengono sistematicamente negati ad alcune categorie di persone e concessi ad altre, trasformando questa tendenza storica in una regola e creando un forte bias (pregiudizio). Un fenomeno simile si osserva nelle ammissioni universitarie, dove i sistemi rischiano di generalizzare i criteri di accettazione avvantaggiando o penalizzando i candidati in base al loro ceto sociale.

1.2.1. Disuguaglianze e BIAS sociali

I dati imperfetti o "rumorosi" possono aggravare enormemente le disuguaglianze. Ad esempio, l'uso di nuovi strumenti IA nel calcolo del rischio di credito può finire per creare svantaggi sistematici per i mutuatari a basso reddito e per le minoranze. Inoltre, i modelli linguistici (word embeddings) possono assorbire i pregiudizi umani e riprodurre pericolosi bias di genere, arrivando a validare associazioni discriminatorie come: "L'uomo sta al Programmatore di computer come la Donna sta alla Casalinga". ProPublica, celebre per aver smascherato bias razziali nei software di previsione della recidiva criminale.

1.2.2. Correlazioni spurie in ambito medico (Artefact Bias)

Un rischio gravissimo deriva dalla fiducia cieca nell'accuratezza dei modelli testati solo su dati storici retrospettivi. Viene portato l'esempio di un sistema di IA progettato per valutare il rischio di complicanze in pazienti affetti da polmonite. Il modello aveva appreso una regola pericolosa: "Se il paziente ha l'asma, allora il rischio è minore".

Questo paradosso era dovuto al fatto che, nella pratica clinica reale, i pazienti asmatici con polmonite venivano immediatamente ricoverati in Terapia Intensiva, ricevendo cure migliori che abbassavano il tasso di mortalità. Il modello aveva scambiato l'intervento medico per una resistenza biologica e, se utilizzato per i triage, avrebbe negato il ricovero a pazienti ad altissimo rischio.

1.2.3. Vulnerabilità dei modelli "Scatola Nera" (Adversarial attacks)

I modelli black box possono essere facilmente ingannati attraverso manipolazioni matematiche impercettibili all'occhio umano. L'aggiunta di un rumore visivo specifico all'immagine di un panda (classificato correttamente con il 57.7% di sicurezza) possa forzare la rete neurale a riconoscere l'animale come se fosse un nematode o, peggio, un gibbono con una confidenza clamorosamente alta del 99.3%.

1.2.4. L'IA che prende "scorciatoie" (Bias di Rappresentazione e nei Dati)

Spesso i modelli raggiungono alti livelli di accuratezza per i motivi completamente sbagliati, imparando a sfruttare dettagli irrilevanti presenti nel set di addestramento:

1. **Husky vs Lupo:** Un classificatore addestrato a distinguere i lupi dagli husky non aveva imparato a riconoscere la fisionomia dell'animale, ma basava le sue decisioni esclusivamente sulla presenza di neve nello sfondo dell'immagine (visto che quasi tutte le foto di lupi nel dataset erano state scattate sulla neve).
2. **Riconoscimento dei cavalli:** In un altro set di dati, un modello classificava correttamente le immagini di cavalli solo perché in quelle foto era presente una filigrana/etichetta ("source tag") nell'angolo in basso a sinistra. Se il tag veniva rimosso, l'immagine non veniva più classificata come cavallo.
3. **Copertine Netflix:** Secondo "The Guardian", le donne vengono mostrate meno in Ads per lavori altamente retribuiti su Google e lo stesso vale per le copertine dei film di Netflix che cambiano a seconda dell'utente.

1.3. AI Act

Il testo normativo dell'AI Act relativo ai sistemi di intelligenza artificiale classificati come *alto rischio*, stabilisce due punti fondamentali:

- Devono essere progettati e sviluppati per garantire un'operatività **sufficientemente trasparente**, così da permettere a chi li utilizza di interpretare gli output e usarli in modo appropriato.
- Devono essere accompagnati da **istruzioni d'uso** che forniscano informazioni chiare, concise, complete e accessibili agli utenti.

Questa figura a forma di piramide rappresenta una classifica di applicazioni reali di AI in quattro livelli di rischio normativo:

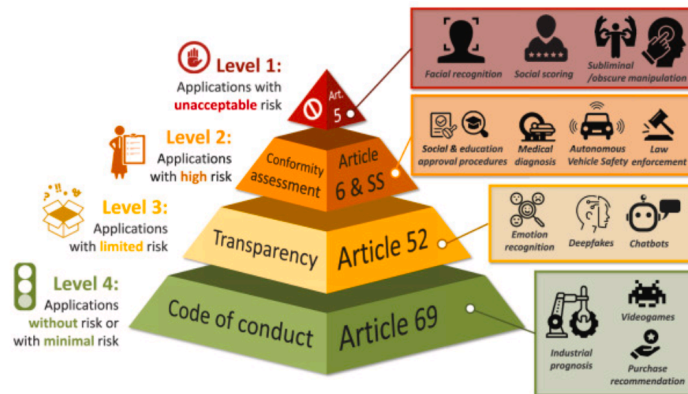


Figura 1: Classificazione dei rischi AI

- **Livello 1:** Rischio inaccettabile. Applicazioni da vietare, come il riconoscimento facciale o il social scoring.
- **Livello 2:** Alto rischio. Settori che richiedono rigide valutazioni di conformità, come le diagnosi mediche, la sicurezza dei veicoli o l'applicazione della legge.
- **Livello 3:** Rischio limitato. Sistemi per i quali è richiesto principalmente un obbligo di trasparenza, come i chatbot.
- **Livello 4:** Rischio minimo. Applicazioni come videogiochi o sistemi di raccomandazione per gli acquisti.

1.4. Interpretabilità vs Explainability

"Ciò che è interpretabile non necessita di essere spiegato."

Alcuni modelli non hanno bisogno di essere **spiegati**, questo perché esistono dei modelli **interpretabili by design**, che sono intrinsecamente interpretabili per via della loro stessa architettura, al contrario delle classiche scatole nere. Alcuni esempi sono: gli alberi decisionali, i modelli lineari generalizzati, sistemi basati su regole, ecc.

1.5. Interpretabilità vs Accuracy

Affrontiamo adesso il dibattito sul presunto compromesso tra le prestazioni di apprendimento (*accuracy*) e la spiegabilità di un modello.

Sebbene esista la convinzione diffusa che i modelli più complessi siano necessariamente meno interpretabili, il documento sottolinea che a volte i modelli interpretabili sono altrettanto accurati. Vengono quindi stabilite delle linee guida pratiche:

- Se un modello interpretabile risulta accurato, **deve essere preferito**.
- L'uso di un modello "scatola nera" dovrebbe essere limitato solo ai casi di reale necessità o quando è l'unico modello disponibile.
- Quando si è costretti a utilizzare un modello black-box, l'obiettivo dovrebbe essere quello di costruire delle **spiegazioni post-hoc**.

1.5.1. Tassonomia degli approcci

Per categorizzare i diversi metodi usati per spiegare le "black-box" vengono usati 3 filoni principali:

1. **Model explanation:** Si tratta di una spiegazione *globale* di un sistema IA opaco. Viene realizzata attraverso un modello interpretabile e trasparente che riesce a catturare completamente la logica del sistema originale. Questo tipo di modelli vengono chiamati anche *metodi globali*.
2. **Model inspection:** Consiste in una rappresentazione visiva o analitica che permette di comprendere alcune proprietà specifiche del modello opaco o della sue previsioni. Non voglio catturare tutta l'essenza del modello ma solo alcune proprietà, qualcosa che vale globalmente per esso ma non una mappa completa.
3. **Outcome explanation:** Invece di cercare di ricostruire l'intero funzionamento del sistema opaco, questo approccio si concentra sul costruire un "esplicatore" in grado di fornire una spiegazione per una **singola istanza specifica**, ovvero il motivo dietro a una specifica decisione o singolo output. Non ambiscono a ricostruire la black box ma cercano una spiegazione ad un certo esempio specifico, come in ambito medico, dove mi interessa la spiegazione del singolo caso non di capire l'intero modello.

2. Bias e Fairness

Il **Bias** è definito come un errore sistematico nei processi decisionali che porta a risultati ingiusti (unfair). Gli algoritmi di ML imparano a *replicare* i bias che sono già presenti nei dati di addestramento.

Il bias è una polarizzazione dei dati o del modello, per questo motivo, se i dati sono sbilanciati è naturale che il modello apprenda questo sbilanciamento. L'XAI ha un ruolo cruciale proprio nell'identificare e affrontare questi bias per garantire decisioni eque.

Questo problema può insorgere in due macro-aree: **Bias nei dati** e **Bias negli algoritmi**.

2.1. Bias nei dati

Quando il problema risiede nei dati con cui il modello viene addestrato, possiamo incontrare diverse tipologie di bias:

- **Sampling Bias (Bias di campionamento):** Si verifica quando i dati non sono raccolti in modo casuale, portando alcuni individui o gruppi ad avere una maggiore probabilità di essere campionati, come nel caso storico dei sondaggi «Literary Digest» per le presidenziali in USA del 1936.
- **Representation Bias (Bias di rappresentazione):** Accade quando il dataset non rappresenta fedelmente la popolazione reale che intende modellare, ignorando o sotto-rappresentando specifiche sotto-popolazioni, come le culture non occidentali nel dataset ImageNet.
- **Artefact Bias (Bias da artefatto):** Nasce da correlazioni spurie tra un elemento artificiale (un artefatto) e le etichette di classificazione. Sostanzialmente sono tag nei dataset che provocano l'effetto Hans, ovvero imparare da dettagli irrilevanti o di sfondo anziché dal soggetto principale.
- **Aggregation Bias (Bias di aggregazione):** Si verifica quando si traggono conclusioni errate sui singoli individui basandosi su considerazioni fatte sull'intera popolazione aggregata. Il caso emblematico di questo bias è il **paradosso di Simpson**.

2.2. Paradosso di Simpson

Il paradosso di Simpson in statistica indica una situazione in cui una tendenza osservabile in diversi sottogruppi di dati scompare o addirittura si inverte quando questi gruppi vengono combinati. Questo accade a causa di una **variabile di confusione** (**confounding variable**), ovvero una terza variabile nascosta che influenza sia la causa che l'effetto.

Esaminiamo due esempi per capire meglio il concetto:

2.3. Trattamento dei calcoli renali

Analizzando i dati aggregati, *il Trattamento B* sembra avere una percentuale di guarigione superiore a quella del *Trattamento A*.

NUMBER OF HEALED CASES	Treatment A	Treatment B
Total	273/350 (78.0%)	289/350 (82.6%)

Tuttavia, se si dividono i pazienti tra chi ha calcoli piccoli e chi ha calcoli grandi, *il Trattamento A* risulta migliore in entrambi i casi. La variabile di confusione è la gravità della malattia: i medici tendevano ad assegnare il *Trattamento B* (meno efficace) ai casi più semplici, falsando le statistiche globali.

NUMBER OF HEALED CASES	Treatment A	Treatment B
Small stones	81/87 (93.1%)	234/270 (86.7%)
Large stones	192/263 (73.0%)	55/80 (68.8%)
Total	273/350 (78.0%)	289/350 (82.6%)

2.4. Mortalità per Covid-19

Il tasso di mortalità totale sembrava più alto in Italia. Tuttavia, analizzando i tassi divisi per fasce d'età, la Cina aveva una mortalità peggiore in quasi tutti i gruppi. La variabile di confusione era l'età della popolazione: l'Italia aveva molti più anziani, che alzavano la mortalità aggregata.

A questo si lega il **Sure-thing principle**, ovvero un'azione che aumenta la probabilità di un evento in ogni sotto-popolazione deve aumentarla anche nella popolazione generale, a patto che l'azione stessa non modifichi la distribuzione delle sotto-popolazioni.

2.5. Bias negli algoritmi

Alle volte i dati sono corretti, ma il bias viene introdotto dalle *meccaniche del modello*:

- **Algorithmic bias**: Avviene quando il bias viene introdotto dall'algoritmo. Ad esempio, un software anti-plagio che analizza lunghe sequenze di testo potrebbe penalizzare ingiustamente le persone non madrelingua inglese.
- **Interaction bias**: Nasce dall'interazione sbilanciata con gli esseri umani. Ad esempio, un chatbot che adatta il suo linguaggio e risponde in modo diverso a uomini e donne.
- **Confirmation bias**: Si manifesta quando un sistema AI viene progettato o utilizzato per confermare credenze e pregiudizi di creatori o utenti. Ad esempio, prevedere il successo di candidati a un lavoro basandosi sui pregiudizi storici dei selezionatori.
- **Generative bias**: Tipico dei modelli generativi, si verifica quando l'output riflette in modo sproporzionato attributi o prospettive presenti nei dati di training, generando contenuti stereotipati o sbilanciati.

3. Fairness

La *Fairness* è definita come l'assenza di qualsiasi pregiudizio o favoritismo verso un individuo o un gruppo basato sulle loro caratteristiche innate o acquisite, i cosiddetti «attributi protetti» come genere o età.

Per valutare la Fairness in termini matematici e probabilistici, esistono diverse metriche rigorose:

- **Equalized Odds**: un predittore $\hat{Y}$ soddisfa questa metrica rispetto all'attributo protetto A e il risultato Y se $\hat{Y}$ ed A sono condizionalmente indipendenti su Y :

$$P(\hat{Y} = 1|A = 0, Y = y) = P(\hat{Y} = 1|A = 1, Y = y), y \in \{0, 1\} \quad (1)$$

Le probabilità di classificare correttamente un positivo (True Positive) e di classificare erroneamente un negativo (False Positive) dovrebbero essere identiche sia per il gruppo protetto che per quello non protetto.

- **Equal Opportunity:** un predittore binario $\hat{Y}$ soddisfa questa metrica rispetto all'attributo protetto A e il risultato Y se:

$$P(\hat{Y} = 1|A = 0, Y = 1) = P(\hat{Y} = 1|A = 1, Y = 1) \quad (2)$$

Richiede che la probabilità di classificare correttamente un esempio positivo sia uguale sia per i membri del gruppo protetto che per quelli del gruppo non protetto. Ci interessa solo che l'esempio positivo sia classificato come tale, indipendentemente dal gruppo di partenza.

- **Demographic Parity (o Statistical Parity):** un predittore $\hat{Y}$ soddisfa questa metrica se:

$$P(\hat{Y}|A = 0) = P(\hat{Y}|A = 1) \quad (3)$$

La probabilità di ricevere un esito positivo dal modello deve essere identica, indipendentemente dall'appartenenza al gruppo protetto.

- Esistono poi altri approcci concettuali come la **Fairness through awareness** (individui simili, calcolati con una metrica di distanza, devono ricevere predizioni simili) e la **Fairness through unawareness** (l'algoritmo è considerato equo finché gli attributi protetti non vengono usati esplicitamente nel processo decisionale).

3.1. Come imporre la Fairness

Per gestire il bias e imporre la fairness si può agire su più livelli:

1. **Pre-processing:** trasformare i dati per rimuovere le discriminazioni intrinseche
2. **In-processing:** modificare gli algoritmi di apprendimento per rimuovere le discriminazioni durante il training (per esempio cambiando la funzione obbiettivo o tramite dei vincoli aggiuntivi)
3. **Post-processing:** utilizzando un validation set dopo la fase di addestramento

4. Valutare l'explainability

Esistono tre principali tipologie di valutazione:

- **Application-grounded evaluation**
- **Human-grounded evaluation**
- **Functionally-grounded evaluation**

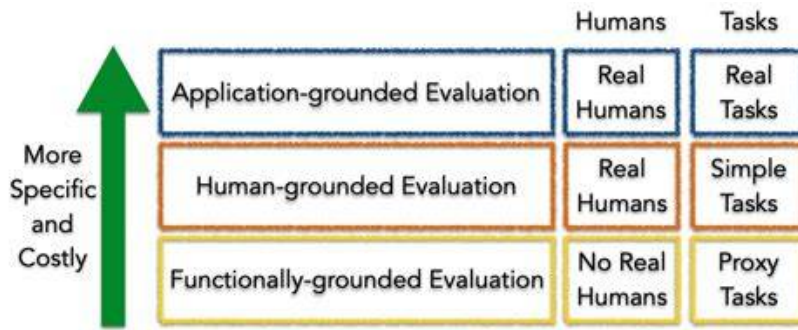


Figura 4: valutazione dell'explainability

4.1. Application-grounded evaluation

Questa tipologia si basa su esperimenti condotti da esperti del dominio su task reali legati all'applicazione (**Real humans, real tasks**). É la valutazione più affidabile ma anche la più costosa.

4.2. Human-grounded evaluation

Basata su umani, non necessariamente esperti, su task semplificate vicine a quelle reali (**Real humans, simplified tasks**). Meno costosa, possibilità di maggiore numero di valutazioni.

4.3. Functionally-grounded evaluation

Non necessita di partecipazione umana (utile nei contesti dove non sarebbe etico) e si basa su metriche proxy (per esempio, **feature importance** attraverso la perturbazione). Nonostante sia un'approssimazione può essere appropriato quando il modello è già stato validato da umani. La difficoltà sta nello scegliere le metriche proxy più adatte alle applicazioni reali e definire le task semplificate.

4.4. Metriche di valutazione

Di seguito vengono elencate alcune metriche di valutazione:

4.4.1. Quantitative performance metrics (col contributo umano)

- **Completeness**: se una spiegazione risulta completa per un utente
- **Simplicity**: si preferisce spiegazioni più semplici
- **Complexity**: tempo richiesto da un umano per comprendere la spiegazione
- **Plausibility**: se una spiegazione risulta persuasiva per un utente
- **Simulability**: se la spiegazione può essere usata per dati nuovi
- **Relevance**: metriche specifiche per il dominio (per esempio clinico o giuridico)

4.4.2. Auxiliary or proxy performance metrics

- **Sensitivity**: quanto un modello è sensibile al valore di una feature
- **Continuity**: esempi simili dovrebbero avere spiegazioni simili
- **Consistency**: quanto una spiegazione è consistente tra modelli diversi
- **Computational cost**: tempo richiesto per calcolare la spiegazione
- **Correctness**: confronto tra spiegazione e una qualche **ground-truth**

- **Fidelity:** misura quanto la spiegazione è allineata all'uscita della black-box

4.5. Approfondimenti sulle metriche

4.5.1. Plausibility (Similarity?)

$$p = \text{similarity}(A^H, A^K) \quad (4)$$

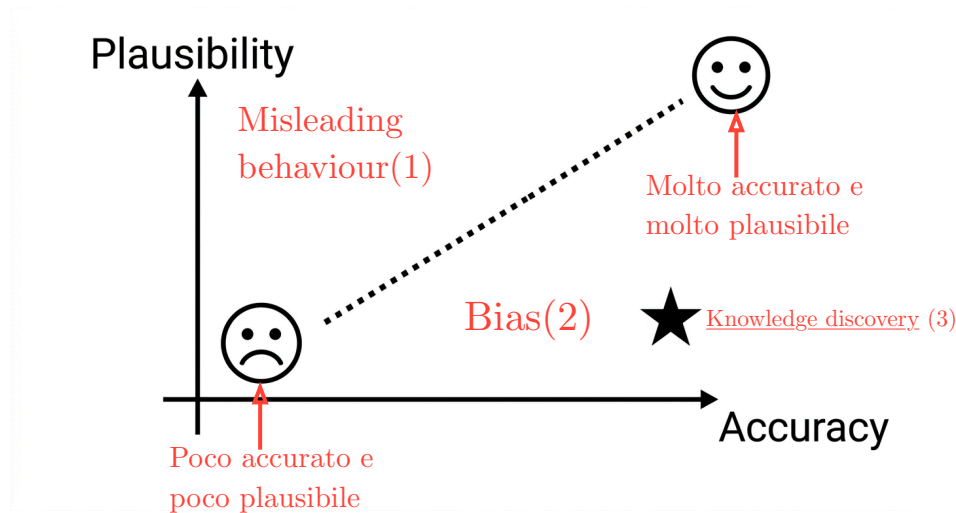


Figura 5: Traduzione da regole logiche a rete neurale

1. Il modello fa una previsione errata ma fornisce spiegazioni forti a riguardo, quindi è «sicuro» di risultato sbagliato
2. Indovina ma per i motivi sbagliati
3. Si scopre qualcosa che non era stato considerato inizialmente, quindi il modello si accorge di associazioni e pattern che non erano noti

4.5.2. Fidelity

Misura l'agreement fra un modello predittivo f e la spiegazione g quando si applica la spiegazione all'input: $f \sim g$. Indica quanto g approssima bene f ma questo non vuol dire che f faccia lo stesso ragionamento che fa g . Nell'explainability è necessario l'utilizzo di più metriche possibili per poter concludere di aver spiegato un certo modello.

4.5.3. Model parameter randomization check

Più che una metrica vera e propria rappresenta un **sanity check**. Per esempio:

Viene preso un modello e ne vengono modificati i parametri (o addirittura re-inizializzati). Se una spiegazione per un certo $\hat{x}$ non cambia, questo significa che non dipende da f e quindi probabilmente dipende da $\hat{x}$ stesso, e questo non va bene. D'altro canto, se la spiegazione cambia non è detto che sia corretta, semplicemente ha passato il sanity check.

4.5.4. Incremental feature deletion

Si tolgono feature incrementalmente (*) e si constata il loro impatto sulla predizione:

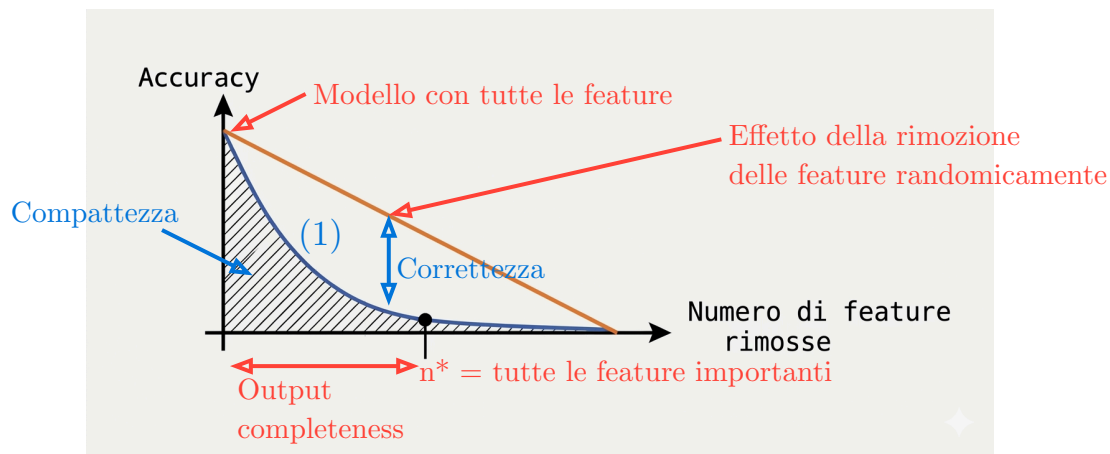


Figura 6: Traduzione da regole logiche a rete neurale

1. Feature rimosse con un criterio di importanza dato dall'explainability delle feature

(*) Come si tolgono le feature?

- Le metto a zero (mascherandole)? Non è proprio il massimo perché a seconda del dominio lo 0 potrebbe essere significativo
- Rimuovo totalmente o ri-addestro? Il ri-addestramento comporterebbe la modifica del modello e quindi sarebbe difficile valutare l'impatto effettivo della feature rimossa
- Riempio con altri valori? Anche con questa possibilità c'è il rischio di usare valori significativi, per cui non è il massimo

5. Data visualization

Nell'ambito dell'interpretazione, è importante osservare i dati e capire la loro distribuzione. Esistono 4 tipi principali di analisi:

- **Analisi univariata:** statistica descrittiva, visualizzazione,...
- **Analisi multi-variata:** correlazioni, interazioni a coppia,...
- **Data distribution:** analisi feature-by-feature, missing values,...
- **Identificazione di outliers e eventuali bias**

5.1. Utilizzo dei grafici

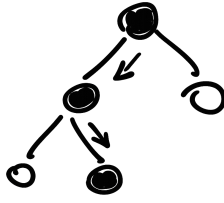
- **Istogramma** Misura la distribuzione (conteggi/frequenze)
- **Grafico a barre:** Utile per le variabili categoriche
- **Scatterplot:** Utile per mostrare la correlazione tra le variabili
- **Boxplot:** Utile a vedere la distribuzione principale e la distribuzione delle «code» (e eventuali outlier)

6. Interpretability by design

Dentro questa categoria sono presenti tutti quei modelli che non hanno bisogno di explainability in quanto l'interpretabilità deriva direttamente dal loro design.

6.1. Decision Tree

I *nodi* sono *attributi/feature*, gli **archi** sono **valori degli attributi** e le foglie sono le classi.



- Ogni **percorso** dalla radice alle foglie sono **regole di classificazione**
- l'albero viene addestrato (learned) **induttivamente** (quindi ricorsivamente) dai dati tabulari

- gli algoritmi di learning sono *greedy* (CART, C4.5), cioè sceglie l'attributo migliore* ad ogni nodo secondo alcuni criteri, senza una funzione di costo globale, quindi una volta scelto un nodo non si può tornare indietro.

* Migliore attributo: bisogna stabilire una metrica es. Purity (Gini index) o Entropy (information gain ratio)

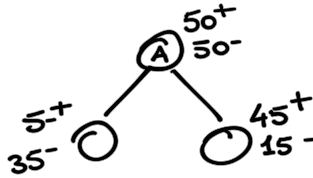


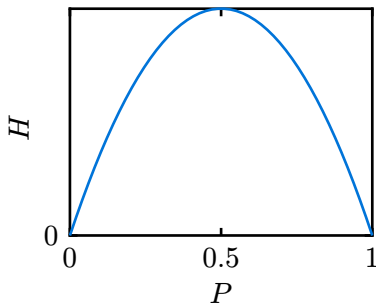
Figura 8: esempio con pesi

- l'attributo ideale divide perfettamente gli esempi negativi e positivi; mentre
- un attributo inutile non modifica la distribuzione tra positivi e negativi

Posso inoltre calcolare una metrica dell'attributo A dalla distribuzione del padre e dei figli, tale metrica può essere l'**entropia** di una distribuzione

$$H(D) = - \sum P_i \log_2 P_i \text{ (Entropia di Shannon)}$$

Dove P_i indica la probabilità (o la proporzione) di esempi appartenenti alla classe i-esima. Questa è una misura del disordine, dunque l'entropia sarà massima quando il disordine è massimo.



l'entropia è massima nel mezzo perché è il punto di una distribuzione che rappresenta la casualità e quindi anche il massimo del disordine in quanto equivale ad andare a caso

$$\text{Gini index} \rightarrow G(A) = \sum_{i=1}^N \rho_i \left(1 - \sum_{k=1}^C \rho_{ik}^2 \right)$$

Il Gini index viene usato come indice di purity, con A attributo, ρ_i come *percentuale di esempi del figlio i-esimo*, ρ_{ik} come *probabilità della classe k del figlio i-esimo*, N numero di figlio di un nodo e C numero classi. Esempio: riprendendo la Figura 8

$$G(A) = \frac{40}{100} \cdot \left(1 - \left(\frac{50}{40} \right)^2 - \left(\frac{35}{40} \right)^2 \right) + \frac{60}{100} \cdot \left(1 - \left(\frac{45}{60} \right)^2 - \left(\frac{15}{60} \right)^2 \right) \quad (5)$$

È un problema di minimizzazione (di $G(A)$), il miglior Gini index è 0.

Se invece abbiamo un attributo numeri continuo? Allora si sceglie una soglia testando tutti i possibili valori dell'attributo nel training set.

6.2. Decision Rules

Sono delle regole in formato **if** condition **then** conclusion.
antecedenteconseguente

L'*antecedente* è formata da un congiunzione di valori delle *feature*, mentre la *conseguente* per la classificazione sarà una *classe*.

Metriche per valutare una decision rules:

- **supporto**: percentuale di casi per cui l'*antecedente* è vero
- **converage (accuracy)**: percentuale di predizioni **corrette** fatte dalla regola, c'è un
↑ supporto ↓ coverage
trade-off tra supporto e coverage: ↓ supporto ↑ coverage
- più «**corto**» è l'*antecedente* (quindi meno condizioni), più **generale** è la regola → alto supporto e bassa copertura
- più «**lungo**» è l'*antecedente*, più **specifica** la regola → basso supporto e alta coertura

Se prendiamo un albero di decisione, overfittare è molto facile, basta espandere l'albero sempre di più finché non arriverò ad un albero molto profondo dove nelle foglie ho un singolo esempio; non si vuole questo ma vogliamo un albero che sia compatto e accurato.

Se abbiamo più alberi di decisione abbiamo dei problemi:

1. le regole possono sovrapporsi → conflitto
2. nessuna regola applicabile → non so decidere

Soluzione:

- decision sets (non ordinati)
- decision list (ordinati)

1. Decision sets → IF condition1 THEN decision1
IF condition2 THEN decision2
⋮
IF none of the above THEN prediction default

In caso di conflitto si fa un voto di maggioranza (solitamente con regole pesate sul supporto)

2. Decision list → IF condition1 THEN decision1
ELSE IF condition2 THEN decision2
⋮
ELSE prediction default

Essendoci l'else non c'è più conflitto ma conta l'ordine, verrà quindi applicata la prima regola che soddisfa la condizione

- sequential covering
- baesian rule lists

Note sulla linearità: sia i decision trees che i decision rules non sono modelli lineari, quindi non è possibile dividere positivi e negativi con un iperpiano; questo si può vedere bene nel grafico in Figura 9.

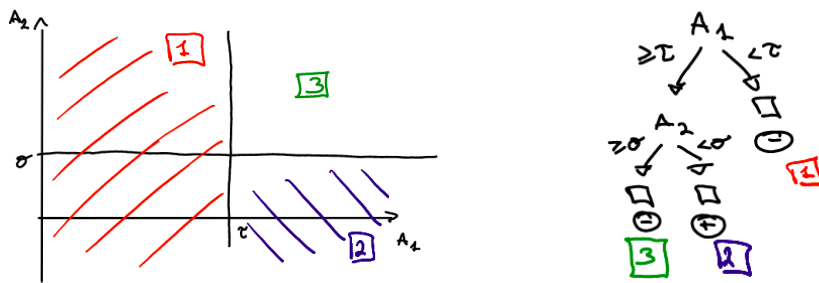
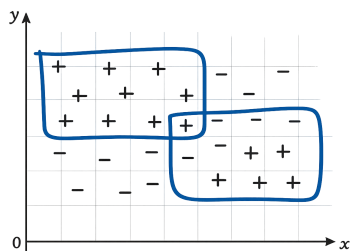


Figura 9: Non linearità decision trees/rules

6.3. Sequential covering



Idea: regole sequenziali che «coprono» esempi differenti da quelli coperti precedentemente

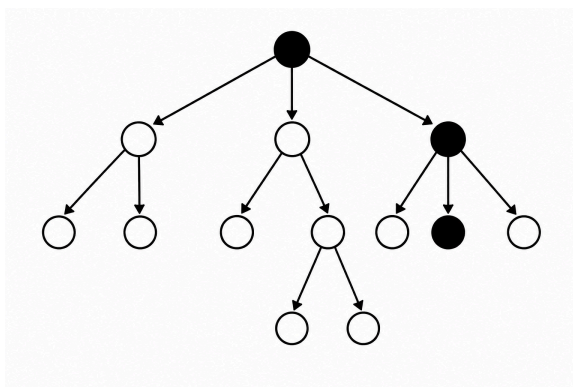
Sketch del Ripper Algorithm:

1. Inizia con una lista L
2. Impara* una regola r
3. Aggiunge r alla lista di regole
4. Rimuovere dal dataset gli esempi coperti da r
5. Se il criterio di stop* non è soddisfatto si torna alla regola 2.

* come si impara una regola di decisione:

- impara un albero di decisione
- dall'albero si estrae la regola con la migliore qualità (purity)

(2.) imparo un albero (ad esempio con CART)



Una regola è un percorso dalla radice alle foglie, la regola viene scelta visitando in maniera ricorsiva l'albero e scegliendo di volta in volta il nodo più puro.

Questo è un algoritmo greedy quindi non c'è criterio globale per scegliere la regola.

Alcuni algoritmi (tipo Ripper) implementano una fase finale di pruning.

★ criterio di stop: - non ci sono più punti da classificare

- soglia sull'accuratezza (per esempio su validation set)

6.4. Bayesian Rule Lists

Liste di regole imparate secondo un **framework bayesiano**, l'idea è di imparare delle liste di regole come **prior**, quando imparo la mia lista scelgo anche quante regole voglio imparare e la lunghezza di queste.

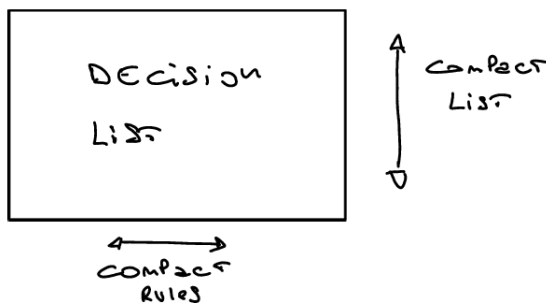
Il metodo, sulla base di ipotesi sulla distribuzione, campiona delle liste e ripetendo il procedimento scelgono la migliore lista con quelle caratteristiche. Fasi principali:

1. estrazione dei pattern frequenti per essere usati come condizioni (antecedenti, non sono regole perché manca la conseguenza es. piastrine alte, età oltre 70) delle regole
→ algoritmo a priori
→ algoritmo fp-growth

Algoritmi usati per scoprire le co-occorrenze tra gli attributi, con l'obiettivo di estrarre pattern frequenti, esempio le condizioni che si verificano più di un certo threshold

2. imparare una lista di decisione da questi pre-minded pattern usando le assunzioni (prior) sulla lista.

Si dà precedenza a liste compatte le cui regole sono compatte, in quanto regole più corte sono meno specifiche e più generali $\Rightarrow$ meno overfitting.



- si genera una lista di decisione iniziale in maniera randomica da una distribuzione a priori Equazione 7
- si modifica la lista aggiungendo, rimuovendo o modificando regole nella lista
- sceglie la lista di decisione migliore secondo una distribuzione a posteriori Equazione 6
- per mitigare la dipendenza della lista iniziale, si ripete la procedura n volte, seguendo un approccio Monte-Carlo e scegliendo alla fine la migliore di tutte.

Distribuzione a posteriori:

$$P(d \mid x, y, A, \alpha, \lambda, \eta) \quad (6)$$

d : decision list target

x : feature

y : target

A : pre-minded target

α, λ, η iperparametri:

- λ = prior sulla lunghezza di d (lunghezza attesa) campionata da una distribuzione poisson con parametro λ

- η = prior sulla lunghezza di una regola (numero atteso di condizioni in una regola), di nuovo da una distribuzione poisson con parametro η
- α = parametri di una distribuzione di Dirichlet multimodali che da' pesi differenti alla classe (con α per ogni classe); solitamente $\vec{\alpha} = 1$

In particolare

$$\overbrace{P(d \mid x, y, A, \alpha, \lambda, \eta)}^{\text{A posteriori}} \propto \overbrace{P(y \mid x, d, \alpha)}^{\text{Likelihood}} \cdot \overbrace{P(d \mid A, \lambda, \eta)}^{\text{A priori}} \quad (7)$$

A posteriori sarà alta se valgono due condizioni:

1. nella *Likelihood* la lista di decisione d classifica correttamente i dati;
2. nella A priori la lista di decisione d ha caratteristiche che matchano la desiderata

Pseudocodice

```

1 pre-mining del pattern A (es. FP-growth)
2 per ogni iterazione j nelle simulazioni Montecarlo
3   | sceglie la lunghezza della decision tree campionando da una Poisson con parametro
   |  $\lambda \rightarrow$  lunghezza m
4   | for i = 1,..., m (imparo m regole)
5   |   | scelgo una lunghezza per l'antecedente della regola  $r_i$  campionando da una Poisson
   |   | con parametro  $\eta \rightarrow$  lunghezza n
6   |   | sceglie dall'insieme A un sottoinsieme di n condizioni
   |   | processo iterativo che modifica le regole (update, add, remove) e calcola la
7   |   | probabilità a posteriori della regola aggiornata, mantenendo la migliore fin'ora
   |   |  $\rightarrow$  migliore decision list per l'iterazione j:  $d_j$ 
8 ritorna la decision list migliore fra tutte:  $d^* = \operatorname{argmax}_j P(d_j)$ 
```

Vantaggi decision list:

- interpretabilità
- espressivi come gli alberi di decisione ma solitamente più compatti
- inferenza veloce
- modelli sparsi (uso di poche feature)

Svantaggi decision list:

- non si usano per fare regressione (sui decision tree invece si può fare regressione)
- tipicamente non si usano feature numeriche
- tipicamente vanno male nell'imparare le dipendenze lineari

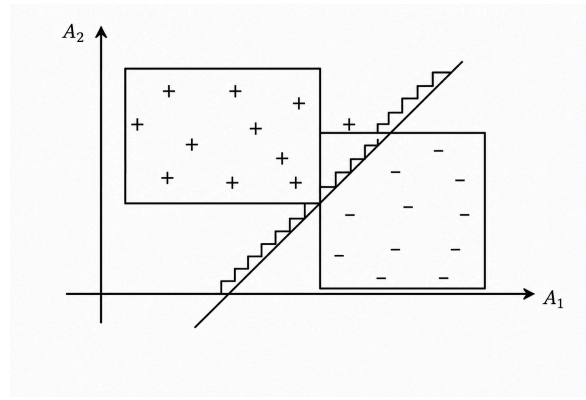


Figura 10: Imparare questi scalini è difficile

6.5. Modelli Lineari

Alta interpretabilità

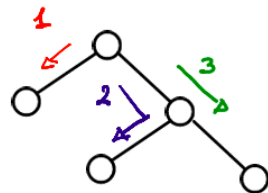
$$f(x) = \sum_{i=1}^P \beta_i x_i + \beta_0 \quad (8)$$

con β_i peso della feature $x_i \rightarrow$ se x_i aumenta di una unità, $f(x)$ aumenta di β_i .

Idea: usare regole come feature con un modello lineare tale che

$$\hat{x}_j \Leftrightarrow \text{la condizione della regola } r_j \text{ vale per } \hat{x} \quad (9)$$

6.5.1. Rule-fit Algorithm



Regola r_1 :	<table><tr><td>cond₁</td></tr></table>	cond ₁	$\Rightarrow$ target ₁
cond ₁			
Regola r_2 :	<table><tr><td>cond₂</td></tr></table>	cond ₂	$\Rightarrow$ target ₂
cond ₂			
Regola r_3 :	<table><tr><td>cond₃</td></tr></table>	cond ₃	$\Rightarrow$ target ₃
cond ₃			

La colonna sarà quindi un nuovo set di feature.

Ogni regola genera una nuova feature booleana, se vale true (1) le condizioni di quella condizione sono soddisfatte, false (0) altrimenti.

Linear model

$$f(x) = \underbrace{\sum_{i=1}^P \beta_i x_i + \beta_0}_{\text{Standard linear model}} + \sum_{j=1}^q \alpha_j r_j \quad (10)$$

Con α_j peso della j-esima regola e r_j la j-esima regola

Idea: introdurre la **non-linearità**, non nel modello, ma nello spazio delle feature **originale**.

Questo si può fare estraendo molte feature, per esempio usando una random forest o un algoritmo gradient boosting. Si va poi a imparare un modello lineare sparso (esempio Lasso)

$$\alpha^*, \beta^* = \underset{\alpha, \beta}{\operatorname{argmin}} \underbrace{\sum_{i=1}^n \mathcal{L}(y_i, f_i(x_i))}_{\text{Classification loss}} + \lambda \underbrace{\left(\sum_{i=1}^p |\beta_i| + \sum_{j=1}^q |\alpha_j| \right)}_{\substack{\text{Penalty term per privilegiare} \\ \text{modelli sparsi (lasso)}}} \quad (11)$$

6.6. Optimal Decision Tree

Nei decision tree la scelta dell'attributo è solitamente **greedy** $\Rightarrow$ non si ha **ottimalità globale**. Creare un decision tree *ottimale* è risaputo essere un problema NP-hard.

$$\text{tree}^* = \underset{\text{tree}}{\operatorname{argmin}} \sum_{i=1}^n \mathcal{L}(\text{tree}, x_i, y_i) + \underbrace{\lambda \# \text{leaves}(\text{tree})}_{\text{Vincolo soft}} \quad (12)$$

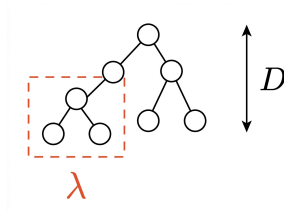
Vincolo (hard) $\text{depth}(\text{tree}) \leq D$ per l'efficienza, mentre il vincolo su λ è soft ed è per la compattezza.

Quindi tra lo spazio degli alberi vado a cercare l'argmin della funzione di costo con i vincoli hard e soft.

$\lambda = 0, D = \infty \rightarrow$

- spazio di ricerca esponenziale
- Consideri solo la loss $\mathcal{L}$

$\lambda > 0, \underbrace{D < \infty}_{D \text{ "small"}} \rightarrow$ La programmazione dinamica rende il problema trattabile



Più l'albero è compatto (quindi meno foglie) più questo è interpretabile (meno regole)

6.6.1. Ingredienti dell'algoritmo

1. Binary data

- categorical: one-hot/ordinal encoding $\rightarrow$

$$\begin{array}{c} \begin{matrix} m \\ \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{matrix} \quad n \quad \begin{matrix} m \\ \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \end{matrix} \quad n \\ \text{One-hot} \quad \text{Ordinato} \\ \text{non ordinato} \end{array}$$

One-hot si usa solitamente dove non ho un ordine, se invece l'ordine è rilevante è necessario andare sulle numerical.

- numerical: discreta con delle soglie $\rightarrow$

in generale ogni possibile valore di queste feature numeriche sono soglie,

Si usa preliminarmente un supporto di alberi per vedere le soglie più usate, così da limitare il numero di feature.

$$\begin{array}{cccc} f_1 & f_2 & f_3 & f_4 \\ \hline & | & | & | & | \\ & < 1.2 & < 2.3 & < 5 & < 1.8 \end{array}$$

le f indicano le feature; in particolare è possibile ridurre le feature utilizzando un threshold guesser (random forest o gradient boosting).

2. Rappresentazioni binarie di problemi da risolvere

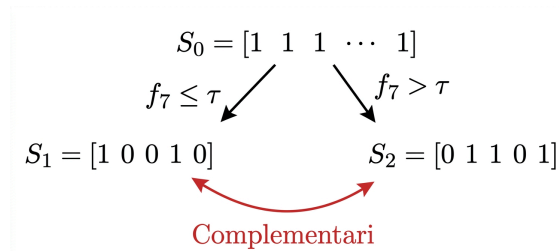
$$S_0 = \underbrace{[1, 1, 1, \dots, 1]}_{\text{\#Training examples}} \quad (13)$$

S_0 rappresenta il **Master Problem**

$S_j[k] = 1$ se il k-esimo esempio di training appartiene al problema

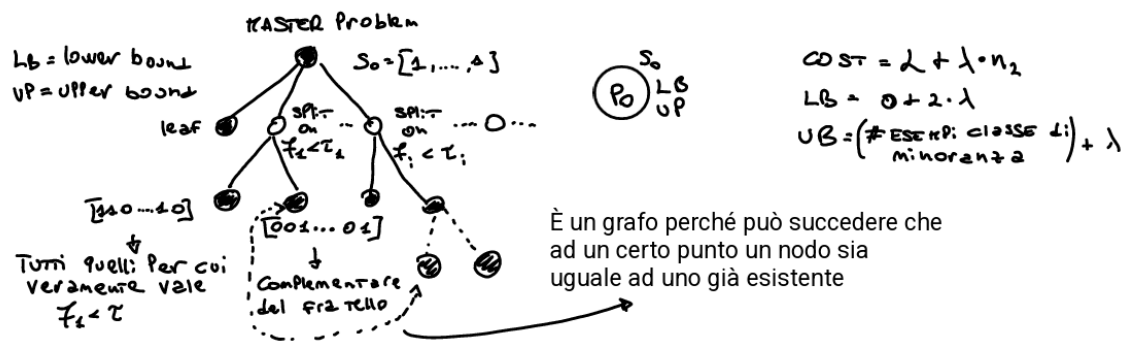
$S_j[k] = 0$ altrimenti

Quindi se un esempio per una certa feature è inferiore ad una certa soglia τ mi va in un set di esempi S_1 se invece è maggiore di τ allora va in S_2 .



3. Code con priorità $Q = \langle S_0, \dots, S_k \rangle$

4. Dependency graph $\rightarrow$ salva i nodi come sottoproblemi



6.7. Gosdt algorithm

(Generalized and Scalable Optimal Sparse Decision Tree)

$S_0 = [111...1]$ MASTER PROBLEM

$G = \emptyset$

$Q = \emptyset$

ADD s_0 TO Q AND p_0 TO G

Algorithm 1: Gosdt algorithm

```

1:  $S_0 \leftarrow [1 \ 1 \ 1 \ \dots \ 1]$ 
2:  $G \leftarrow \emptyset$ 
3:  $Q \leftarrow \emptyset$ 
4:  $Q \leftarrow Q \cup s_0$ 
5:  $G \leftarrow G \cup p_0$ 
6: while  $p_0.LB \neq p_0.UB$  do
7:    $\triangleright$  Continua finché il problema master non è stato risolto
8:   Estrai  $s$  da  $Q$ 
   Trova il  $p$  in  $G$  associato a  $s$  o crea un nuovo nodo in  $G$  (calcolando LB e UB)
9:   if  $p.LB = p.UB$  then
10:    continue Questo problema è stato risolto!
11:   end
12:   for Each feature  $j$  do
13:     Crea i figli  $s_l^j$  e  $s_r^j$ 
     Trova/aggiungi  $p_l^j$  e  $p_r^j$  in/a  $G$ 
     Aggiungi a  $Q$  soltanto i figli non ancora risolti (1) o non da sfoltire (2)
14:   end
15:   Aggiorna upper e lower bound per  $p$  e i suoi genitori *
16:   if  $p.LB = p.UB$  then
17:    continue Questo problema è stato risolto!
18:   end
19:   for Each feature  $j$  do
20:     Aggiungi a  $Q$  soltanto i figli non ancora risolti (1) o non da sfoltire (2)
21:   end
22: end
23: return  $G$  Da  $G$  è possibile estrarre l'albero di decisione ottimo!

```

6.7.1. Le condizioni 1 e 2

1. $p_l^j.LB = p_l^j.UB$
 $p_r^j.LB = p_r^j.UB$ Quindi questo è già stato risolto (LB = UB)
2. $p_l^j.LB + p_r^j.LB > p.UB$ Se, splittando, otterrei qualcosa di peggiore del genitore, non aggiungere i nodi a Q

6.7.2. Come si aggiornano il lower e upper bound *

Tra tutte le feature j salviamo la somma minima di LB/UB tra p_l e p_r :

$$1 \quad LB' = \min_j (p_l^j.LB + p_r^j.LB) \quad (14)$$

$$2 \quad UB' = \min_j (p_l^j.UB + p_r^j.UB) \quad (15)$$

3 Se $LB' \neq p.LB$ oppure $UB' \neq p.UB$ (quindi aggiorna solo se diversi)

4 | (A) $p.UB = \min(p.UB, UB')$

5 | (B) $p.LB = \min(p.UB, \max(p.LB, LB'))$

6.7.2.1. Intuizione per A

L'*upper bound* considera il caso peggiore, quindi $p.UB$ non può essere maggiore del più piccolo tra i figli (UB').

6.7.2.2. Intuizione per B

Il *lower bound* considera il caso migliore per cui $p.LB$ non può essere inferiore al più piccolo tra i figli ma non può nemmeno essere maggiore dell'*upper bound*.

6.8. Generalized Additive Models (GAM)

→ Caso speciale: Explainable Boosting Machine (EBM)

$$f(x) = \sum_{i=1}^{P \leftarrow \text{numero di feature}} \beta_i x_i + \beta_0 \text{ (Linear model)} \quad (16)$$

Classificazione → Regressione Logistica $y = f(x) = \sigma\left(\sum_{i=1}^P \beta_i x_i + \beta_0\right)$

$$g(y) = \sum_{i=1}^p \beta_i x_i + \beta_0 \text{ Generalized Linear Model} \quad (17)$$

$$\underbrace{g(y)}_{\substack{\text{Link} \\ \text{function}}} = \sum_{i=1}^P \overbrace{f_i(x_i)}^{\text{Shape functions}} \quad (18)$$

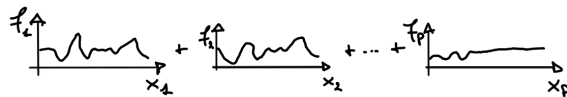
Ogni funzione f_i è appresa da una *feature* che quindi costituisce un contributo additivo alla funzione target.

L'Equazione 18 è molto **interpretabile**, in quanto le f_i sono di un solo parametro e quindi anche facilmente plottabili, nel caso in cui non ho assunzioni particolari e queste sono anche fortemente non lineari, dipende dalla scelta di f .

Posso ottenere sia spiegazioni locali che globali!

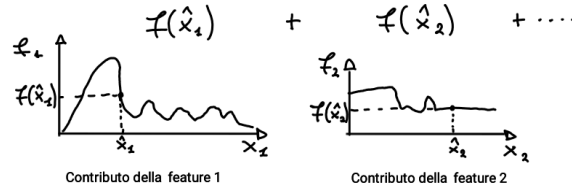
1. Globale:

$$g(y) = \sum_{i=1}^P f_i(x_i) = \quad (19)$$



Così posso vedere come ogni feature contribuisce al risultato finale

2. Locale:



Limiti del modello: non considera l'interazione tra le feature $f(x_1) + f(x_2) \neq f(x_1, x_2)$.

Estensione GA2M: considerare coppie di feature:

$$g(y) = \sum_{i=1}^P f_i(x_i) + \underbrace{\sum_{i=1}^P \sum_{j=i+1}^P f_{ij}(x_i, x_j)}_{\substack{\text{Aggiunge una funzione per le} \\ \text{coppie di feature (se sono} \\ \text{troppe si selezionano solo le} \\ \text{"più promettenti" scondo} \\ \text{alcune euristiche)}}} \quad (20)$$

Dato che ora siamo su due dimensioni e non più una, allora l'interpretabilità si ha attraverso delle heatmap

6.9. Explainable Boosting Machines

Le **Explainable Boosting Machine** rappresentano una specifica implementazione dei modelli **GAM**. In questo approccio, ogni funzione f_i associata a una determinata feature è modellata attraverso un *tree ensemble* (un insieme di alberi decisionali).

$$\underline{\text{ENSEMBLE}} = \left\{ \begin{array}{l} \text{Molti classificatori} \\ \text{Molti regressori} \end{array} \right\} \text{ Weak learners} \quad (21)$$

RANDOM FOREST = Modelli appresi in parallelo e il modello finale è la media o votazione a maggioranza)

$$\begin{array}{ccc} \text{Albero 1} & \text{Albero 2} & \text{Albero 3} \\ \text{A} & \text{A} & \text{A} \end{array} \rightarrow \text{Votazione a Maggioranza} \quad (22)$$

GRADIENT BOOSTING = Modelli appresi sequenzialmente correggendo gli errori (residui) dei precedenti. Il modello finale è una combinazione pasata di alberi.

$$\text{Albero 1} \rightarrow \text{Albero 2} \rightarrow \text{Albero 3} \rightarrow \text{Media Pesata} \quad (23)$$

Questi modelli rientrano nella categoria dei modelli «*Glassbox*» (intrinsecamente interpretabili), poichè non sono scatole nere da interpretare a posteriori, ma il loro stesso design permette di seguirne la logica. In un modello **GAM** come le **EBM**, il risultato finale è una scomposizione additiva delle funzioni di ogni singola feature:

$$\text{EBM} = f_1(x_1) + f_2(x_2) + \dots + f_p(x_p) \quad (24)$$

6.9.1. Meccanismo di training

Le EBM utilizzano un algoritmo di apprendimento per regressione basato sui residui, in cui un insieme di classificatori o regressori deboli, viene addestrato sequenzialmente. Il training avviene secondo un processo iterativo e ciclico (round-robin) per isolare gli effetti:

- **Inizializzazione:** Si parte impostando la funzione di forma a zero per tutte le feature $f_i = 0$ per ogni feature j .
- **Ciclo iterativo (Round robin):** Per un numero prefissato di iterazioni M , il modello scorre sequenzialmente ogni feature j da 1 a P , con P numero di features.
- **Calcolo dei residui:** Per la specifica feature in esame, si definisce un set di addestramento

$$\mathcal{R} = \left\{ \left(x_{i,j}, y_i - \sum_{k \neq j} f_k \right) \right\} \quad (25)$$

- **Aggiornamento additivo:** Viene appreso un singolo albero che funge da *Shape function* $\mathcal{S}$ sui residui. L'aggiornamento della funzione per la feature avviene tramite la formula matematica:

$$f_j = f_j + \eta \cdot \mathcal{S} \quad (26)$$

dove η è il **learning rate**.

- **Prevenzione della collinearità:** Se il *learning rate* η viene mantenuto sufficientemente piccolo, l'ordine in cui le feature vengono analizzate e aggiornate nel ciclo perde d'importanza, rendendo il modello stabile.

Per correttezza visualizziamo anche l'intero algoritmo: $f_j = 0$, η **learning rate**

Algorithm 2: Algoritmo di apprendimento EBM (Regressione)

```

1:  $f_j \leftarrow 0$ 
2: while  $p_0.LB \neq p_0.UB$  do
3:   if  $p.LB = p.UB$  then
4:     continue Questo problema è stato risolto!
5:   end
6:   for Each feature  $j$  do
7:     Crea i figli  $s_l^j$  e  $s_r^j$ 
       Trova/aggiungi  $p_l^j$  e  $p_r^j$  in/a  $G$ 
       Aggiungi a  $Q$  soltanto i figli non ancora risolti (1) o non da sfoltire (2)
8:   end
9:   if  $p.LB = p.UB$  then
10:    continue Questo problema è stato risolto!
11:  end
12:  for Each feature  $j$  do
13:    Aggiungi a  $Q$  soltanto i figli non ancora risolti (1) o non da sfoltire (2)
14:  end
15: end
16: return  $f(x) = \sum_{j=1}^P f_j(x_j)$  Modello finale additivo interpretabile

```

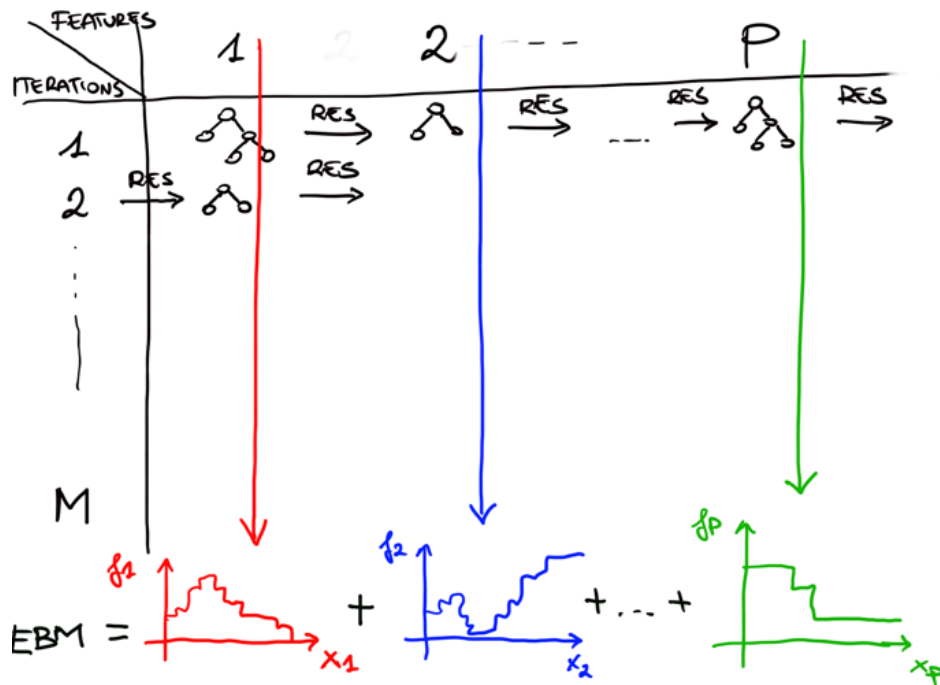


Figura 17: Evoluzione delle features all'aumentare delle iterazioni

In generale, gli ensemble methods combinano vari modelli: le Random Forest usano alberi appresi in parallelo calcolando la media (o il voto di maggioranza), mentre il Gradient Boosting apprende gli alberi in modo sequenziale per correggere i residui precedenti, generando una combinazione pesata finale. Nelle EBM, le «shape functions» isolate permettono di tracciare grafici a gradini o continui per visualizzare in modo trasparente l'impatto di ogni specifica feature (come $x_1, x_2, \dots, x_p$) sull'output, rendendo il modello completamente «aperto» agli occhi umani.

6.9.2. Neural Additive models (cenni)

I **Neural Additive Models (NAMs)**, introdotti intorno al 2020 da studiosi come Caruana e Hinton, rappresentano un'evoluzione dei modelli additivi in cui le «shape functions» (funzioni di forma) f_j non vengono apprese tramite insiemi di alberi decisionali (come nelle EBM), ma vengono apprese attraverso l'uso di **Reti Neurali (Neural Networks)**.

Dettagli tecnici sul loro funzionamento:

- **Architettura di base:** Ogni singola feature x_i viene processata separatamente da una propria rete neurale dedicata, la quale calcola e restituisce in output la relativa funzione di forma $f_i(x_i)$.
- **La funzione di attivazione ExU:** Al posto delle classiche funzioni di attivazione comunemente usate nel Deep Learning (come la ReLU), i NAMs utilizzano una funzione di attivazione innovativa chiamata **ExU**.
- **Formula e logica della ExU:** La formula matematica alla base di questa attivazione è $\sigma(x) = e^{w(x-b)}$, dove la caratteristica fondamentale è che il peso (w) si trova posizionato all'esponente. Lo scopo di questa funzione è far sì che piccole differenze nei dati di input producano grandi differenze nell'output. Questo stratagemma permette

alla rete neurale di ricreare visivamente quel comportamento «a gradini» (steps) tipico dei grafici generati dalle EBM classiche.

- **Gestione delle interazioni a coppie:** Anche i NAMs possono supportare un'estensione per considerare le interazioni tra coppie di variabili (*pairs of features*). A livello architetturale, questo si traduce semplicemente nell'utilizzo di reti neurali progettate con due unità di input anziché una.

6.10. Prototype-based Neural Network

Intro → Metodo pensato per le immagini, una parte dell'architettura è una rete neurale. Si basa sul ragionamento umano legato al modo in cui un umano descriverebbe un'immagine.

Metodo KNN (K-nearest neighbours): stimato in base alla distanza

↓ Instance of (27)

Case-based reasoning: ragionamento basato su casi conosciuti

↓ Inspiration for (28)

Prototype-based Neural Network

Ben performante sulle immagini ma l'interpretabilità è basata sui vicini (n prototipi)

→ Convolutional neural network $f(W_{\text{conv}})$

→ Prototype layer g_p (g_{pj} learnable parameter)

→ Fully connected layer $h(W_h)$

$f(x)$ convolutional output $H \times W \times D$

Con $H \times W$ **dimensione dei filtri** che voglio imparare ($\ll$ img. size) relativamente piccoli e D **iperparametro** (quanti filtri voglio imparare)

L'idea è di imparare m (iperparametro) prototipi delle immagini

$$P = \{P_j\}_{j=1}^m \quad \text{Patch} \quad (29)$$

$$m = m_k \times \underbrace{k}_{\# \text{classi}} \quad (m_k \text{ prototipi per ogni classe})$$

Ogni patch P_j ha dimensione $\underbrace{H_1}_{\leq H} \times \underbrace{W_1}_{\leq W} \times D$

Una «patch» nell'output convoluzionale

Una «parte» dell'oggetto da classificare.

Inferenza

Supponiamo di averla addestrata, come funziona l'inferenza?

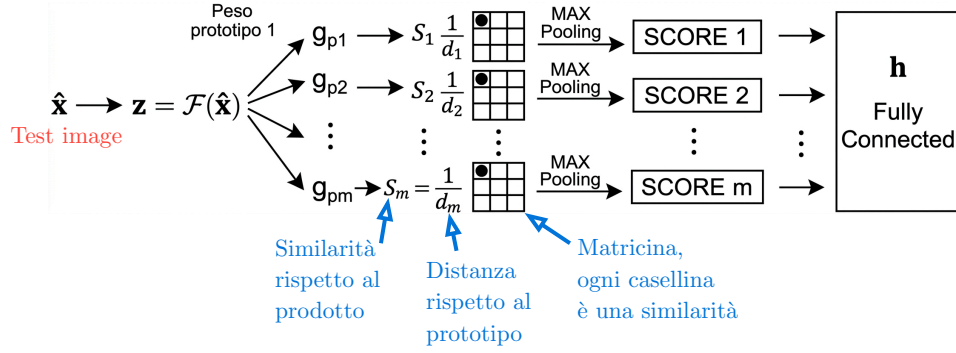
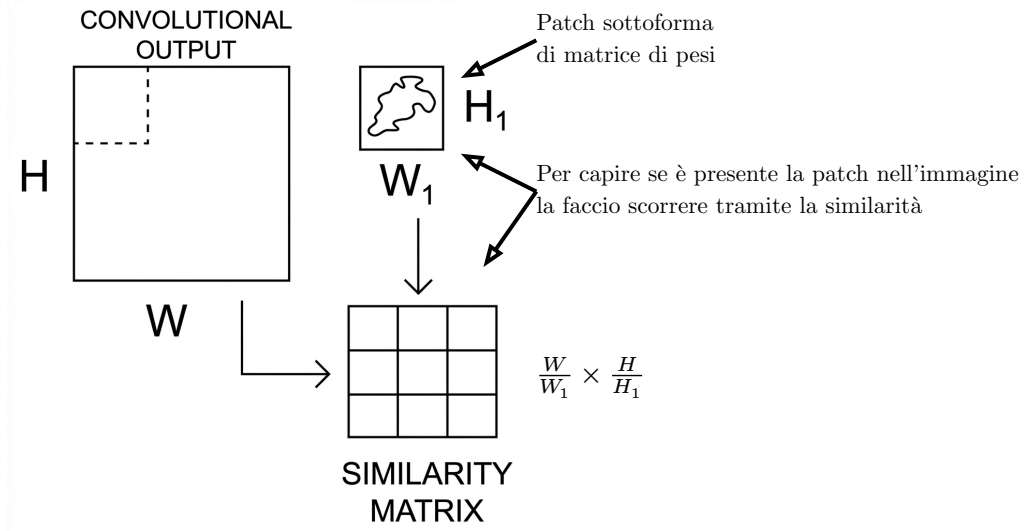


Figura 18: Sarebbe $S_1 = \frac{1}{d_1}$ e $S_2 = \frac{1}{d_2}$



In pratica la spezzettiamo in pezzi grandi $H_1 \times W_1$ e confrontiamo questi pezzi con P_1 .

Training (si imparano insieme)

- Iterativamente
 1. Joint learning di f e g
 2. Prototype projection
 3. learning di h

Nel dettaglio:

1. Stochastic gradient descent per tutti i layer eccetto h (W_h «frozen»), ottimizzazione congiunta W_{conv} e $\{P_j\}_{j=1}^m$

loss function:

$$\min_{P_j, W_{\text{conv}}} \frac{1}{n} \sum_{i=1}^n \underbrace{\mathcal{L}(h \cdot g_P \cdot f(x_i), y_i)}_{\text{loss di classificazione}} + \underbrace{\lambda_1 \cdot \text{clst} + \lambda_2 \cdot \text{sep}}_{\text{Regolarizzazione}} \quad (30)$$

Cross-entropy loss

$$\text{clst} = \frac{1}{n} \sum_{i=1}^n \min_{j: P_j \in P_{y_i}} \min_{z \in \text{patches}(f(x_i))} \|z - p_j\|_2^2 \quad (P_j \text{ è un filtro}) \quad (31)$$

$f(x_i)$ è l'output della rete convoluzionale di cui prendo tutte le patch dell'immagine e guardo quella più vicina ai prototipi di classe j e tra quelle prendo la minima

$$\text{sep} = \frac{1}{n} \sum_{i=1}^n \min_{j: P_j \notin P_{y_i}} \min_{z \in \text{patches}(f(x_i))} \|z - p_j\|_2^2 \quad (32)$$

Serve per massimizzare la distanza rispetto alle altre classi
Prototipi per le classi diverse da y_i

2. Prototype projection *

$$P_j \leftarrow \min_{z \in Z_j} \|z - P_j\|_2^2 \text{ dove } Z_j = \{\tilde{z} \in \text{patches}(f(x_i)) \mid \forall i \text{ t.c. } y_i = k\} \quad (33)$$

Prototipo della classe k
Classe

* C'è un teorema che garantisce che la proiezione non degrada le performance (fino ad un certo livello)

3. Ottimizzazione convessa sull'ultimo layer h

$$\min_{W_h} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(h \cdot g_P \cdot f(x_i), y_i) + \lambda \sum_{k=1}^K \sum_{j: P_j \notin P_k} |W_h^{(k,j)}| \quad (34)$$

$W_h^{(k,j)} \approx 0$ per i prototipi non di classe k

6.11. ILP - Programmazione logica induttiva

La **Programmazione Logica Induttiva** o **ILP** affonda le sue radici nella programmazione logica, che è un paradigma computazionale di tipo **dichiarativo** e non imperativo. Ci si limita quindi a descrivere i fatti e le regole del problema, senza dover specificare alla macchina la sequenza esatta di istruzioni per risolvere un problema. In ambito ILP, l'obiettivo principale è partire da un «predicato bersaglio» (*target predicate*) che vogliamo comprendere o prevedere, per riuscire a trovare/apprendere una **teoria logica in grado di spiegarlo**. In termini semplici, l'ILP è una forma di machine learning «glassbox» in cui il modello finale che viene appreso dalla macchina non è una funzione matematica o una rete neurale, ma un vero e proprio programma logico facilmente leggibile dall'uomo.

Cenni alla sintassi e alle definizioni di base:

1. Variabili: si indicano con **lettere maiuscole**.
2. Costanti: si indicano con **lettere minuscole**.
3. Predicati e Arità: Anche questi in minuscolo. L'arità indica il numero di argomenti che il predicato accetta (es. father/2 rappresenta il predicato «padre» che collega 2 elementi).
4. Termini: possono essere una variabile, una costante, oppure una funzione composta da n argomenti.
5. Atomi: sono formule scritte come $p(t_1, \dots, t_n)$, dove p è il predicato e i vari t sono i termini. Esempi di atomi sono father(X, Y) o father(X, alice).
6. Atomi Ground: Sono atomi «fissati» che non contengono variabili, ma solo costanti (es. father(bob, alice)).
7. Letterali: Un letterale è un atomo o la sua negazione logica, spesso introdotta da not, esempio not father(bob, alice).
8. Clausole (o regole): Hanno una struttura divisa in **testa** (head) e **corpo** (body). Si presentano nella forma $h_1, \dots, h_m :- b_1, \dots, b_n$, dove tutti gli elementi sono letterali e il simbolo $:-$ significa «se». Un insieme di clausole forma una **teoria**.

9. *Clausole di Horn*: Sono un tipo particolare di clausole caratterizzato dall'avere un solo letterale positivo, che deve trovarsi nella testa della regola. Una clausola di Horn priva di variabili è definita *Ground Clause*.

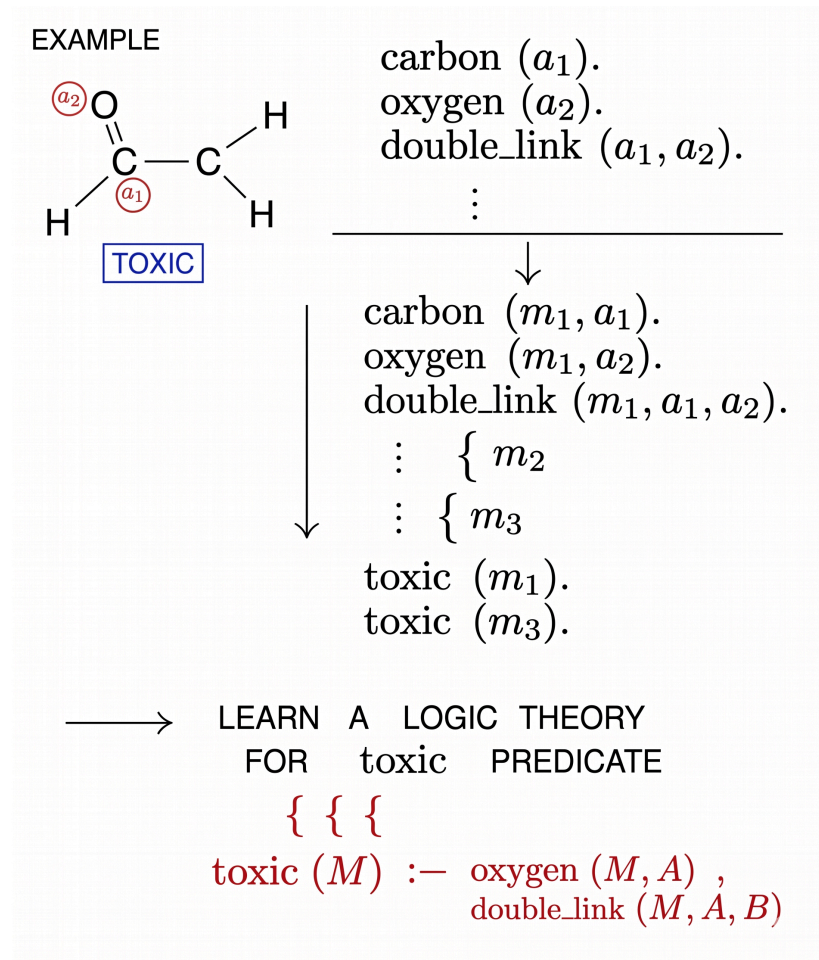


Figura 20: Evoluzione delle features all'aumentare delle iterazioni

6.11.1. Semantica dell'ILP

Per comprendere il ragionamento automatico, l'ILP si basa su una rigorosa semantica formale legata alla logica di Herbrand:

1. **Universo di Herbrand**: È l'insieme di tutte le possibili costanti (cioè senza variabili, ma solo valori concreti) del mio dominio che possono essere formati utilizzando le costanti e le funzioni del dominio. Esempio: {alice, bob, carl, david}.
2. **Base di Herbrand**: È l'insieme di tutti i possibili atomi «ground» che si possono costruire applicando i predicati disponibili agli elementi dell'Universo di Herbrand. Esempio: {happy(alice), happy(bob), healthy(alice), healthy(bob)}
3. **Interpretazione di Herbrand**: Consiste nell'assegnare un valore di verità a ciascun elemento presente nella base di Herbrand.
4. **Closed-world assumption**: Nelle interpretazioni viene applicata questa assunzione fondamentale, per cui vengono specificati solo i fatti veri. Tutto ciò che non è esplicitamente elencato nell'interpretazione viene automaticamente considerato falso.

5. **Entailment (Implicazione logica $\models$)**: Una teoria T implica logicamente una clausola C ($T \models C$) se e solo se ogni modello che rende vera la teoria T rende vera anche la clausola C . L'obiettivo dell'ILP, ad esempio, è imparare una teoria (es. $\text{happy}(X) :- \text{healthy}(X), \text{passed_exam}(X)$), che implichi e spieghi i fatti che osserviamo es. $\text{happy}(\text{alice})$

6.11.2. Metodologie di Learning

Esistono due impostazioni principali attraverso le quali un sistema ILP può imparare dalle osservazioni:

- **Learning from Entailment (LFE)**

E' il setting più classico. Si ha a disposizione:

1. Una **Knowledge Base (B)** o Background Knowledge, contenente fatti e clausole di base.
2. Un insieme di esempi positivi (E^+) e negativi (E^-) estratti da una singola interpretazione globale.
3. **Obiettivo**: Trovare un'ipotesi logica H all'interno di uno spazio di ricerca $\mathcal{H}$ tale che, unita a B , implichi logicamente tutti gli esempi positivi e non implichi nessuno degli esempi negativi.

Completezza: $\forall e \in E^+, H \cup B \models e$

Consistenza: $\forall e \in E^-, H \cup B \not\models e$

- **Learning from Interpretation (LFI)**

Gli esempi positivi e negativi non sono singoli fatti, ma veri e propri *insiemi di fatti* derivanti da **interpretazioni diverse** (es. ogni interpretazione è un paziente diverso o una molecola separata).

1. **Obiettivo**: Trovare un'ipotesi H tale per cui gli esempio (E^+) «soddisfino la teoria» (ogni $e \in E^+$ deve essere un modello per $H \cup B$), mentre gli esempi negativi (E^-) non la soddisfino.
2. *Integrazione*: Il LFE è più indicato quando si analizzano grossi grafi interconnessi, mentre il LFI è tipicamente usato quando i dati possono essere facilmente divisi in frammenti indipendenti e non collegati.

6.11.3. Caratteristiche di un sistema di ILP

Qualsiasi software o algoritmo di ILP deve definire quattro componenti fondamentali:

1. **Learning setting**: Quale approccio usare (LFE, LFI, o altri meno usati)
2. **Representation language**: Il linguaggio formale scelto per descrivere sia la Background Knowledge che l'Ipotesi, inclusi limiti o vincoli espressivi.
3. **Language bias**: Un set di regole cruciali che definiscono e restringono «lo spazio delle ipotesi» ($\mathcal{H}$). Senza un language bias, lo spazio delle regole logiche generabili all'infinito sarebbe troppo vasto per poter essere esplorato da una macchina.
4. **Search method**: L'algoritmo di ricerca utilizzato per navigare questo spazio alla ricerca della regola migliore.

6.12. 4. Il Sistema Aleph

Sviluppato intorno al 2001, Aleph è un classico algoritmo di ILP che utilizza un approccio iterativo bottom-up per apprendere le regole. Il processo si divide in 4 fasi cicliche:

1. **Selezione:** Seleziona a caso un esempio positivo dal dataset che deve essere generalizzato.
2. **Costruzione della Bottom Clause:** Crea la clausola logica più specifica possibile che sia coerente con il *language bias* e che implichi l'esempio scelto.
3. **Ricerca e Valutazione:** Cerca una clausola più generale rispetto alla **Bottom Clause**. Per capire se la nuova regola è buona, le assegna uno score calcolato come:

$$S = P - N \quad (35)$$

dove P sono gli esempi positivi coperti e N quelli negativi coperti erroneamente.

4. **Aggiornamento:** Aggiunge la clausola con lo score migliore all'ipotesi finale H . A questo punto rimuove dal dataset tutti gli esempi positivi ormai coperti (spiegati) da questa nuova clausola e torna al punto 1, ripetendo il ciclo finché non restano più esempi positivi da spiegare.

Integrazione: Questo meccanismo di aggiungere regole e rimuovere i dati già spiegati è noto nel Machine Learning come approccio di copertura (*Sequential/Set Covering*). Aleph è tutt'oggi ampiamente studiato per la sua versatilità basata su Prolog.

6.13. 5. Il problema dei treni di Michalski

Integrazione sul contesto storico: Proposto dal ricercatore Ryszard Michalski alla fine degli anni '70, questo è in assoluto uno dei *toy-problem* (problemi giocattolo) più celebri nella storia del machine learning simbolico, utilizzato come test diagnostico per valutare i sistemi esperti.

- **Il problema:** Il problema mostra due insiemi di treni: alcuni viaggiano verso Est (*Eastbound*) e altri verso Ovest (*Westbound*). I treni hanno caratteristiche diverse (numero di vagoni, tipo di tetto, forma del carico, numero di ruote).
- **L'obiettivo:** Imparare la regola logica (Target Predicate: `eastbound(train)`) in grado di classificare correttamente cosa accomuna tutti i treni diretti a Est.
- **Dati:** La macchina ha a disposizione fatti base (Background Knowledge, B) per descriverli, come ad esempio: `has_car(train, car)`.
`has_three_wheels(car).`
`no_roof(car).`

Risultato (Ipotesi Logica H):

L'ILP è in grado di estrarre e rendere leggibile una regola complessa, espressa in Prolog come:

```
eastbound(X) :-
    has_car(X, Y), no_roof(Y),
    has_car(X, Z), has_three_wheels(Z),
    Y \= Z.
```

7. Approcci neurosimbolici

7.1. Simbolico vs Connessionista

- **Simbolico**: ragionamento basato sulla manipolazione formale dei simboli.
Caratteristiche:
 - Fondato sui principi della logica
 - Sfrutta delle conoscenze di base
 - Altamente interpretabile
 - Richiede discretizzazione per l'utilizzo
- **Connessionista (sub-simbolico)**: ragionamento basato sul processare unità logiche (semplici) interconnesse. Caratteristiche:
 - Gestisce meglio le conoscenze incerte
 - Non richiede la codifica delle conoscenze
 - Necessita di una grande collezione di dati
 - Tipicamente, gli algoritmi basati su questi approccio sono più facilmente distribuibili
 - Da un punto di vista dell'interpretabilità, solitamente viene visto come una «black box»
 - Ha un graduale decremento delle prestazioni quando l'input è rumoroso

Gli **approcci NeuroSimbolici(NeSy)** combinano la logica con l'approccio neurale.

7.2. Approcci pioneristici: KBANNs

Sta per **Knowledge Based Artificial Neural Networks**. È uno dei primi tentativi di iniettare conoscenza nelle **ANN(Artificial Neural Networks)**, interpretando i modelli come regole logiche.

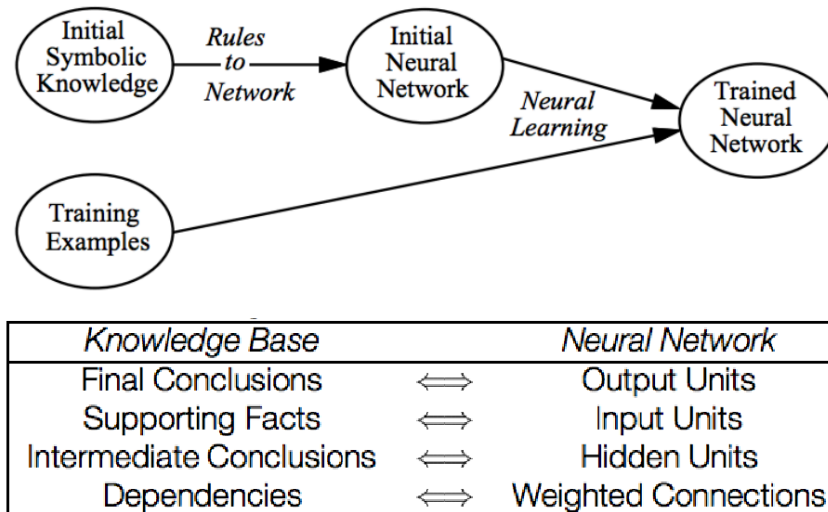


Figura 21: Workflow generale e corrispondenze KB - NN

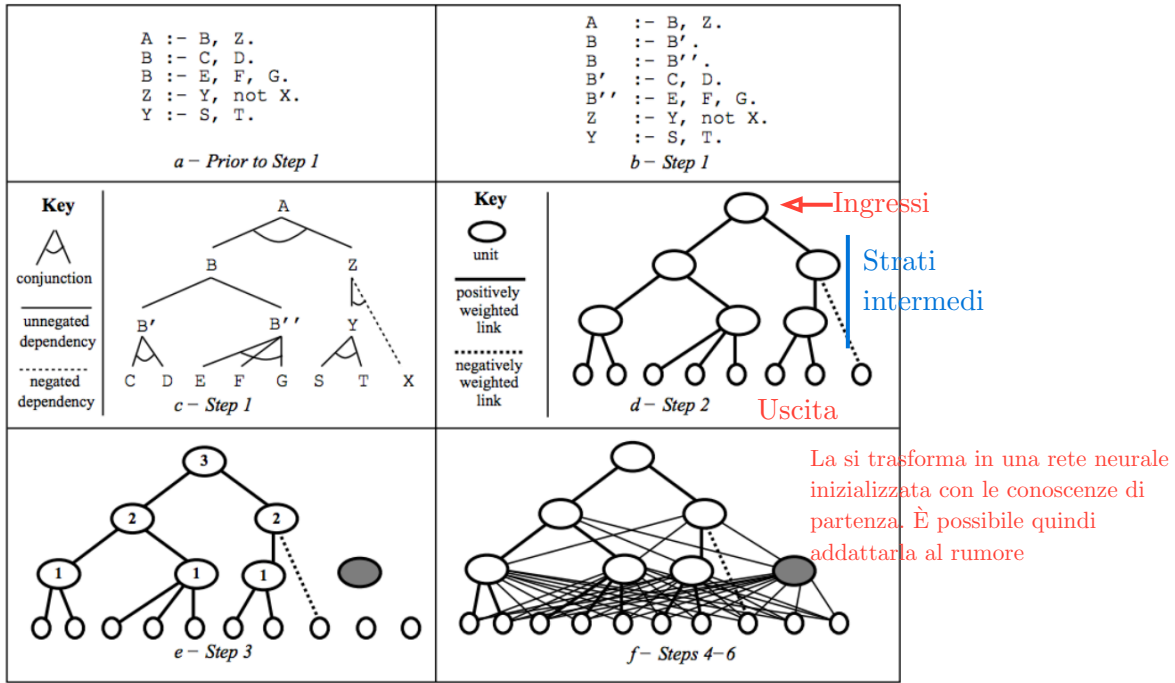


Figura 22: Traduzione da regole logiche a rete neurale

7.3. NeSy e SRL/StarAI

Questi sono approcci più recenti.

- **Statistical Relational Learning (SRL)**: combina logica con apprendimento probabilistico.
- **Neural-Symbolic Learning and Reasoning (NeSy)**: combina logica con la neuroscienza cognitiva.

7.3.1. SRL/StarAI

Mira a combinare la **logica del I ordine** e i **modelli logici** per l'apprendimento e il ragionamento:

- Sfrutta il potere espressivo della logica del I ordine
- Gestisce le incertezze n tramite i modelli grafici
- Unisce la logica e l'inferenza (probabilistica)

7.3.2. Nesy

Mira a combinare i **modelli neurali** con gli **approcci simbolici**:

- Codifica la conoscenza nell'architettura della rete
- Usa un termine di regolarizzazione per codificare le regole
- Vincola le computazioni dei neuroni tramite regole

La logica può essere usata come:

- un **programma neurale**, cablandola direttamente nell'architettura della rete (l'idea nasce da KBANN).
- un **regolarizzatore**: combinando la loss di classificazione standard con una loss che considera i vincoli logici:

$$Loss = \text{ClassificationLoss} + \lambda \text{SemanticLoss} \quad (36)$$

Come funziona la logica integrata nella rete neurale?

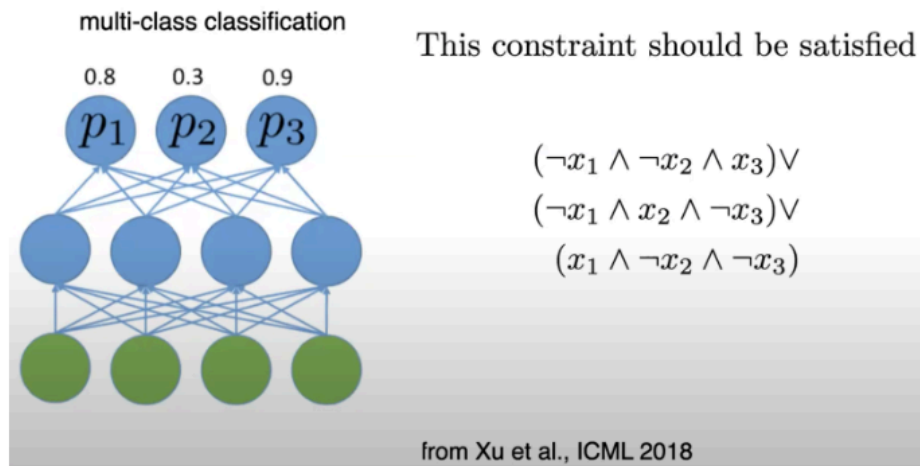


Figura 23: Vincolo logico di classificazione

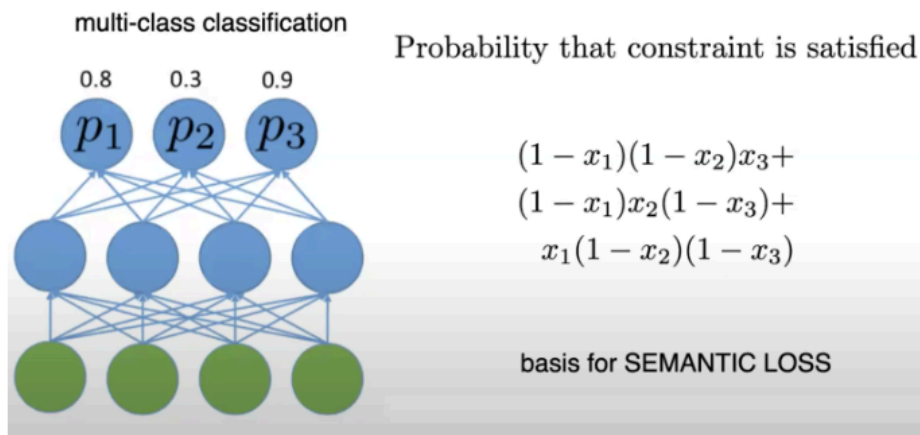


Figura 24: Probabilità legata all'architettura del modello che il vincolo venga soddisfatto

Quindi le regole (**logica**) vengono tradotte in una **penalty**(loss): i letterali x li metto come $1 - x$ se negati e come x se non negati.

7.4. Classic SRL/StarAI tasks (problems)

Le applicazioni di StarAI devono interfacciarsi con tre problematiche distinte ma fortemente interconnessi

- inferenza
- parameter learning
- structure learning

7.4.1. Inference

L'inferenza in StarAI si colloca nell'intersezione tra *inferenza logica* e *inferenza probabilistica*

7.4.1.1. Logical inference

Dedurre il valore di verità di alcuni fatti logici, data una collezione di altri fatti e regole.
Se parliamo di logica pura ci aspettiamo qualcosa del genere:

```
likesmovie(X,M) :- moviegenre(M,G), likesgenre(X,G).
likesmovie(X,M) :- friends(X,Y), likesmovie(Y,M).
```

Nota: è necessario effettuare il grounding (*cioè il processo di sostituire le variabili (X, M, G,...) con costanti concrete per ottenere fatti/regole)

```
likesmovie(alice,bladerunner).
friends(alice,bob).
likesgenre(alice,sciencefiction).
...
```

Se assegno una **probabilità** alle varie **regole** allora entro nel campo della statistica, quindi non ho più solo la logica pura. le principali tecniche sono **model-based** come il *SAT solver* oppure **proof-based** che cerca le *derivazioni*.

In ogni caso il **bottleneck** rimane il grounding, che è un problema perché se ho tante regole l'inferenza diventa estremamente costosa; esistono **tecniche** per evitare di generare *tutti* i predicati oppure *esplorare* simmetrie e pattern/template; nei casi in cui l'inferenza resta molto costosa si può ricorrere all'*inferenza approssimata*.

7.4.1.2. Probabilistic inference

Dedurre la distribuzione a posteriori di variabili casuali non osservate, date quelle osservate

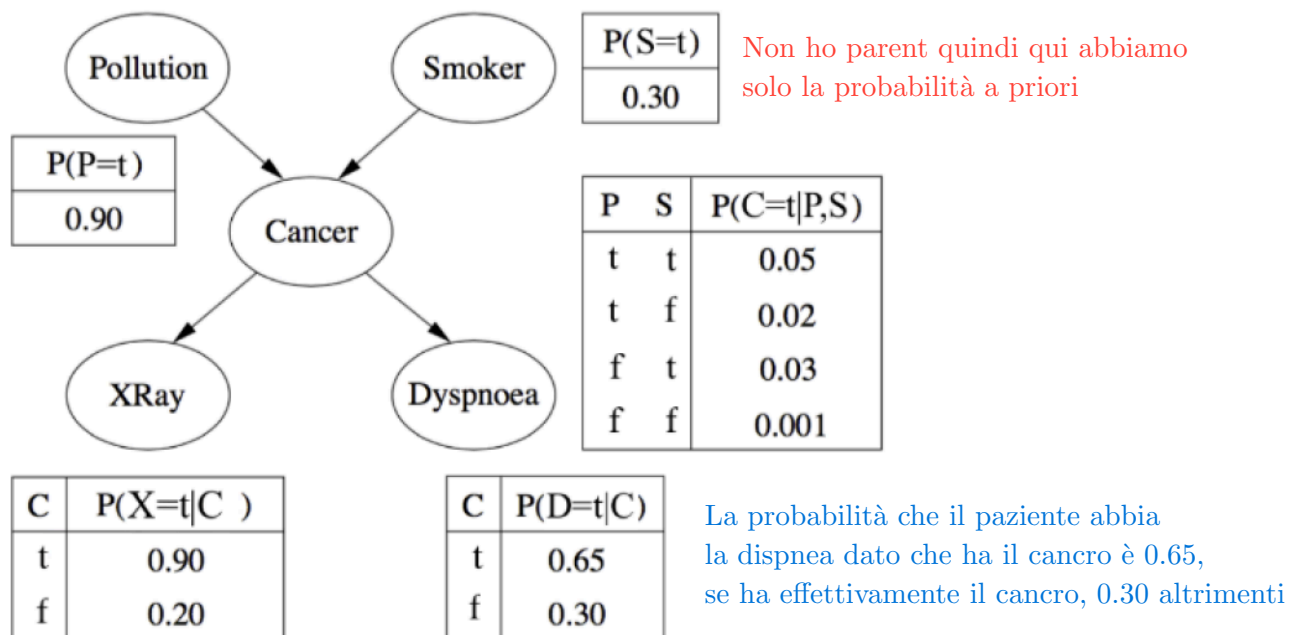


Figura 25: Esempio di Bayesian Network

Nota: la freccia non indica la **causa** (causazione) ma una **dipendenza**.

7.4.2. Parameter Learning

Tipicamente, modelli come StarAI specificano un set di **parametri** (probabilistici o reali) associati a regole/clausole per modellare l'*incertezza nella conoscenza disponibile*. Questi

parametri possono essere imparati dai dati attraverso: tabelle di probabilità di reti Bayesiane; pesi o probabilità associate a regole soft.

7.4.3. Structure Learning

Problema impegnativo: imparare direttamente le regole (la struttura) del modello, ci sono più approcci differenti:

- imparare parametri e regole congiuntamente
- imparare prima le regole (es. con ILP) e poi i loro pesi

7.4.4. Collective Classification

Questo framework può essere facilmente sfruttato per eseguire **classificazione collettiva** in un set di **esempi non indipendenti**, come nodi in un grafo, frasi in un documento, ... Dato un insieme di regole (possibilmente neurali), e una collezione di costanti/feature che rappresentano il documento, l'algoritmo di inferenza computa il **mondo più probabile**, o interpretazione, assegnando così un valore di verità a ciascun predicato presente nel documento.

Confronto fra i tre metodi: l'**inferenza** mi permette dato un insieme di regole stabilire **valori di verità** a livello logico, in **parameter learning** date le regole imparare il **peso** che hanno, mentre in **structure learning** imparo anche le **regole**, quindi fare anche ILP.

7.5. Modelli e framework

7.5.1. Markov Logic Networks (MLN)

Al contrario dei **vincoli hard** imposti dalla logica, la logica di Markov sfrutta dei **vincoli soft**.

Una **Markov Logic Network** è composta da:

- Un insieme di formule del I ordine
- Un insieme di pesi, ciascuno associato ad una formula

In questa formulazione, è improbabile che una regola venga violata ma non è impossibile, al contrario della logica pura dove la formula corrisponde ad una verità assoluta.

La sintassi è opposta rispetto a Prolog: le costanti sono definite col maiuscolo mentre le variabili col minuscolo.

Maggiore è il peso associato alla regola più è probabile un mondo in cui la regola risulta vera. La semantica delle MLNs induce una **distribuzione di probabilità** su tutti i possibili mondi. Indicando con $\mathcal{X}$ un insieme di variabili aleatorie rappresentate nel modello, abbiamo:

$$P(\mathcal{X} = x) = \frac{\exp\left(\sum_{F_i \in \mathcal{F}} w_i n_i(x)\right)}{Z} \quad (37)$$

dove $n_i(x)$ è il numero dei fatti veri di F_i nel mondo x e Z è la funzione di partizione:

$$Z = \sum_{x \in \mathcal{X}} \exp\left(\sum_{F_i \in \mathcal{F}} w_i n_i(x)\right) \quad (38)$$

È un **setting discriminativo**: solitamente, alcuni atomi sono sempre **osservati** (evidence X), mentre gli altri sono **sconosciuti** al momento della predizione (query Y). La probabilità di un mondo/di una certa configurazione dipende dai **pesi** w_i e dal numero di **fatti** n_i di ogni formula F_i :

$$P(Y = y \mid \mathcal{X} = x) = \frac{\exp\left(\sum_{F_i \in \mathcal{F}} w_i n_i(x, y)\right)}{Z_{\mathcal{X}}} \quad (39)$$

L'inferenza punta a trovare la più probabile y data x :

$$y^* = \operatorname{argmax}_y P(Y = y \mid \mathcal{X} = x) \quad (40)$$

7.5.1.1. Inferenza

Dato un insieme di fatti noti, le regole pesate possono essere usate per inferire il valore di verità di altri fatti (**query**). Essendo un problema **#P-completo** è possibile usare algoritmi approssimati.

Un problema comune in inferenza logica (probabilistica) è che tipicamente è necessario effettuare il **grounding** (*Sezione 7.4.1.1) della base di conoscenza sui predicati. Questo può diventare velocemente insostenibile sia da un punto di vista di memoria sia di tempo. Learning

Sia i **pesi** che le **regole** stesse possono essere **imparate** da una collezione di osservazioni di predicati, andando a **massimizzare** condizionalmente la **log likelihood** di una query di predicati **dati i fatti** (l'inferenza funge da sottoroutine).

$$\frac{\partial}{\partial w_i} \log P(Y = y \mid X = x) = n_i - E_w[n_i] \quad (41)$$

Cioè il gradiente rispetto al peso della regola è il numero di grounding di quella regola a cui sottraggo il valore atteso, facendo l'inferenza, dei grounding con i pesi attuali w

7.5.2. ProbLog

È un'estensione probabilistica di **Prolog** in cui è possibile associare probabilità a fatti e regole. **DeepProbLog** estende ProLog computando le probabilità tramite una rete neurale.

7.5.3. MLN vs ProbLog

- In una **MLN**, il peso della formula F rappresenta i log-odds tra un mondo in cui F è vera e uno in cui F è falsa
- In **ProbLog** viene direttamente specificata la probabilità che una regola risulti vera

Riassunto: ho una parte percettiva che non posso demandare totalmente alla logica, quindi ho bisogno di una rete neurale che assorba e quindi risolva questa mancanza. Se però sono in un dominio dove ho una base di conoscenza (che viene da vincoli sulle classi, o feature che legano le classi), negli approcci neurosimbolici riesco a codificare la base di conoscenza e integrarla al mio problema così che questo poi funzioni meglio dell'approccio puramente neurale.

8. Spiegazioni Post-Hoc (Spiegazioni a posteriori)

Il concetto alla base di questi metodi è quello di **spiegare a posteriori** le decisioni di una black-box. / Si possono evidenziare tre problemi principali:

1. **Model Explanation ~ Globale** (Spiegazione globale del problema, es. TREPAN)
2. **Outcome explanation ~ Locale** (Spiegazione dell'outcome, la singola predizione, es. LIME, SHAP)
3. **Model Inspection ~ Ibrido** (Misure statistiche sulla black box che danno un'idea di come funziona il modello, es. Feature Importance)

1. Model Explanation

Dato un predittore di *black box* b e un *dataset* X , il problema della **model explanation** consiste nel trovare una spiegazione $E \in \varepsilon$ che appartiene ad un dominio ε human-interpretable attraverso un predittore **globale** interpretabile $C_g = f(b, x)$. Una spiegazione E è ottenibile da C_g se $E = \varepsilon_g(C_g, X)$ per qualche spiegazione logica ε_g .

Si parte da una black box e da un dataset X , che spesso non è il training set, e voglio replicare il comportamento della black box, che è molto diverso (a volte anche più semplice) da imparare una rete neurale in un certo task; la black box magari è bravissima nel suo task ma non sarebbe altrettanto brava se usassimo un modello interpretabile, ma far un sistema che replica il comportamento a quel punto sarebbe diverso.

2. Outcome Explanation

Dato un predittore di *black box* b e una singola istanza x , il problema dell'**outcome explanation** consiste nel trovare una spiegazione $e \in \varepsilon$ che appartiene ad un dominio ε human-interpretable attraverso un predittore **locale** interpretabile $C_l = f(b, x)$. Una spiegazione e è ottenibile da C_l se $e = \varepsilon_l(C_l, x)$ per qualche spiegazione logica ε_l .

Quindi essenzialmente per ogni istanza x mi devo imparare un modello, magari un modello leggero (o magari no) però c'è da dire che comunque il costo computazionale non è trascurabile.

3. Model Inspection

Dato un predittore di *black box* b e un dataset X , il problema di *model inspection* consiste nell'individuare una rappresentazione (visuale, testuale o altro) $r = f(b, x)$ di qualche proprietà di b usando una qualche procedura f .

Essenzialmente vogliamo costruire delle statistiche, quindi avere una rappresentazione della black box da un punto di vista della *feature importance*. Quindi non voglio qualcosa che mi dica né perché ho classificato qualcosa in un certo modo né mi dica esattamente la logica di predizione della black box, ma mi sappia dire quali feature sono più importanti e quali magari invece non lo sono, così se quelle feature non sono mai importanti posso decidere di toglierle.

Per classificare questi modelli di spiegazione ci sono delle **tassonomie** basate su:

- Tipologia di problema (classificazione, regressione ecc.)

- Tipo di spiegazione*
- tipo di blackbox
 - agnostiche
 - specifiche (oppure un mix, metodi agnostici ma specializzati in un modello specifico)
- tipo di dato
 - immagini (mappe di salienza)
 - testo (attention)
 - tabulare

* tipi di spiegazione:

- Alberi di decisione
- Decision rules
- Feature importance
- Modelli lineari
- Mappe/maschere di salienza
- Analisi di sensitività
- Partial dependency plots
- Prototipi
- ⋮

8.1. Model Explanation

Global Explanation → Idea

- Interrogare la black box per raccogliere predizioni (che sono dati di test per la black box ma dati di training per l'explanator)
- allenamento sul dataset
- controllare che il modello imparato è *allineato* (fidelity: MSE, accuracy, roc rispetto alle label predette) alla blackbox $C_g \sim b$

(Non voglio che C_g sia estremamente accurato, se sbaglia va bene ma deve sbagliare come b , quindi deve essere fedele a b e non magari fare meglio)

TREPAN [1996]

- Global explanation
- Neural network (ma agnostiche)
- Alberi di decisione* come spiegazione
- dati tabulari
 - * Tecnicismi nel training degli alberi di decisione: es. dare più importanza a esempi che sono più significativi (chiamati prototipi)

Sui metodi globali ci sono anche altre idee: se è facile trovare una spiegazione di qualche modello black box si può partizionare i dati e imparare modelli che sono interpretabili (Palm 2017)

8.2. Outcome Explanation

Local explanations.

- Modelli surrogati locali (LIME)
 - Consistono nel trovare un predittore interpretabile nelle «vicinanze» dell'esempio che voglio spiegare

- Feature perturbation o removal (PreDiff, SHAP?) idea: cambio o rimuovo una feature e osservo l'importanza dall'uscita
- Saliency masks/map (GradCAM, IntGradients)
idea: evidenziare porzioni dell'input che sono più influenti per la predizione (parole o frasi in un testo o parti di un immagine)

8.3. LIME - Local Interpretable Model-agnostic Explanations

[Rebeiro 2016]

A noi interessa una **local explanation** di uno dei positivi, di tale prendo un intorno e questo sarà un'istanza di test da spiegare.

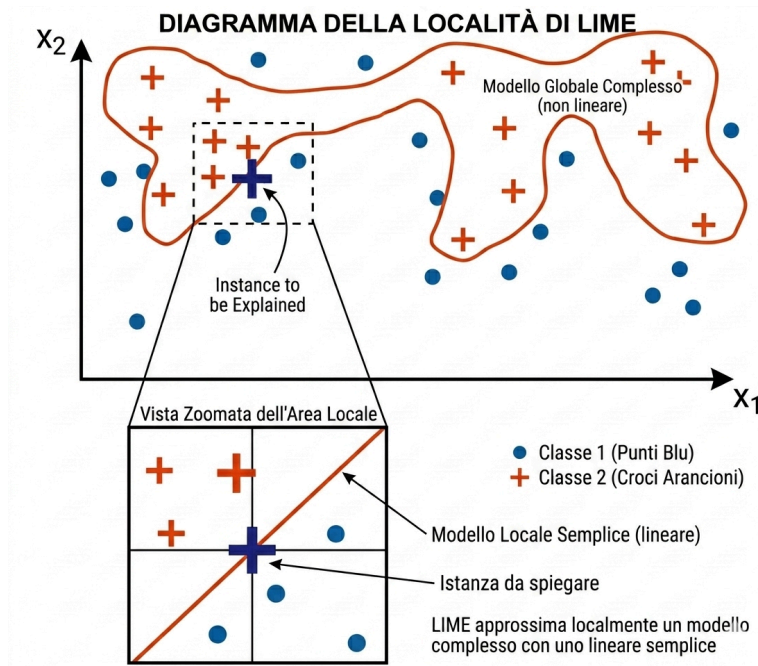


Figura 26: Esempio semplificato con sole due feature

Idea:

- imparare un **modello interpretabile locale** in prossimità della predizione da spiegare
- **generare campioni** per imparare il modello locale **perturbando** l'istanza di test $\hat{x}$
- scegliere una **rappresentazione** per $\hat{x}$ che sia interpretabile. Vogliamo una local fidelity (come il modello si comporta in prossimità dell'istanza da spiegare) che **non** implichi una global fidelity

Voglio quindi costruire

$$\text{explanation}(x) = \underset{g \in G}{\operatorname{argmin}} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (42)$$

Funzione di prossimità
Black box
Istanza da spiegare Famiglia di modelli interpretabili Quello che voglio trovare
Comlessità di g . Regularizzatore

Con $\mathcal{L}$ che assicura la **fidelity** locale (più mi avvicino a x più voglio che g sia fedele) e Ω mi assicura l'interpretabilità

8.4. Ingredienti di LIME

1. Necessito di definire una rappresentazione interpretabile

$$\underbrace{x \in \mathbb{R}^d}_{\text{Rappresentazione originale}} \rightarrow \underbrace{x' \in \{0, 1\}^{d'}}_{\text{Rappresentazione interpretabile}} \quad (43)$$

Es. per il testo, $x \in \mathbb{R}^d$ potrebbe essere *embedding*
 $x' \in \mathbb{R}^{d'}$ potrebbe essere presenza/assenza di parole (bag of word)

Es. per le immagini, $x \in \mathbb{R}^d$ RGB pixel
 $x' \in \mathbb{R}^{d'}$ presenza/assenza di super pixel

2. Proximity function π_x che definisce la località nell'intorno x Funzione di distanza
(distanza coseno per il resto
LR distance per immagini)

$$\pi_x(z) = \exp\left(\frac{-D(x, z)^2}{\sigma^2}\right) \quad (44)$$

Iperparametro
(quanto voglio essere rigido
o tollerante rispetto alle istanze)

3. Loss (local fidelity) function

$$\mathcal{L}(f, g, \pi_x) = \sum_{\substack{z, z' \in Z \\ \text{Vicini} \\ \text{(campioni)}}} \pi_x(z) \cdot \underbrace{(f(z) - g(z'))^2}_{\text{Fidelity}} \quad (45)$$

Le z sono gli esempi nello spazio di partenza, z' sono gli esempi nello spazio interpretato e quindi ho bisogno di un meccanismo per passare da uno all'altro. Facendo lo scarto quadratico sto misurando la **fidelity** pesata per la **prossimità**, la fidelity degli esempi vicini mi interessa molto mentre la fidelity di quelli lontani mi interessa meno.

4. Regolarizzazione (complessità)

$$\Omega(g) = \infty \cdot \underbrace{\mathbb{I}\left[\|W_g\|_0 > k\right]}_{\substack{\text{Funzione} \\ \text{indicatore}}} \quad \left. \begin{array}{l} \text{Iperparametro} \\ \# \text{feature} \\ \# \text{parole} \\ \# \text{superpixel} \end{array} \right\} \begin{array}{l} \text{Al massimo} \\ \text{nella spiegazione} \end{array} \quad (46)$$

8.5. Lime Algorithm (Sparse Linear Explanations)

Richiede: $\underbrace{f}_{\text{Black box}}$, $\underbrace{N}_{\text{Numero esempi}}$, $\underbrace{x}_{\text{Test instance}}$, $\underbrace{x'}_{\text{Interpretable test instance}}$, $\underbrace{\pi_x}_{\text{Proximity measure}}$, $\underbrace{k}_{\text{Lunghezza spiegazione}}$

$$\operatorname{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (47)$$

Sketch dell'algoritmo

Algorithm 3: Algoritmo di LIME

```

1:  $Z \leftarrow []$ 
2: for  $i = 1, \dots, N$  do
3:    $z_i \leftarrow \text{samplearound}(x')$ 
4:    $Z \leftarrow Z \cup \{z_i, z'_i, \pi_x(z_i)\}$ 
5: end
6:  $w \leftarrow \text{K-Lasso}(z, k)$   $\left. \begin{array}{l} z'_i \text{ as feature} \\ f(z_i) \text{ as target} \end{array} \right\} \text{linear model } g$ 

```

Come si vede in $w \leftarrow \text{K-Lasso}(z, k)$ la k influisce sulla regolarizzazione dell'Equazione 47. Samplearound:

$$\begin{array}{c}
 x' : \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \\
 \downarrow \\
 z'_i = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 1 & 1 \end{bmatrix}
 \end{array} \tag{48}$$

Inoltre bisogna essere sicuri di poter andare da $x \rightarrow x'$ e viceversa $x' \rightarrow x$.

Tutta questa procedura serve per generare una **singola** text explanation.

Es. Black box = Random forest con 1000 alberi

$N = 5000 \rightarrow 3$ secondi per spiegazione

Black box = Inception Network (rete neurale)

$N = 5000 \rightarrow 10$ minuti per spiegazione

Dato che il tempo degli utenti (esperti) è limitato, ci sono approcci per scegliere gli esempi più rappresentativi da mostrare.

Es. *Submodular-pic (PC) algorithm*

Sceglie l'istanza di test più rappresentativa da mostrare all'utente basandoci su un certo budget B (tempo).

	f_1	f_2	f_3	f_4	
x_1	■				$x_1 \dots x_5$ sono <i>test instance</i> mentre $f_1 \dots f_4$ sono <i>feature</i>
x_2		■			Le celle bianche corrispondono alle feature che Lasso ha scartato.
x_3		■	■	■	In questo caso verrebbero scelti gli esempi 3 e 5, in quanto il 3 è quello che copre più feature, il 5 copre comunque un buon numero di feature e una di queste non è coperta dall'esempio 3.
x_4		■		■	Essenzialmente fa maximum coverage delle feature importanti
x_5	■	■			$\rightarrow$ quindi risolvere un problema di ottimizzazione

8.6. PredDiff (Prediction difference)

Il **Prediction difference** è un metodo di spiegazione post-hoc locale, progettato per spiegare le singole previsioni di modelli black box.

8.6.1. Spiegazione tramite rimozione

L'intuizione alla base è molto semplice, ovvero, per capire quanto una specifica caratteristica (feature A_i) sia importante per la classificazione di una specifica istanza x , calcoliamo la differenza nell'output del modello prima e dopo aver «rimosso» quella feature. Se la previsione cambia drasticamente, significa che la feature rimossa era ovviamente importante.

8.6.2. Formulazione matematica

Presentiamo le due metriche principali che stanno alla base del funzionamento del metodo:

1. PredDiff

- Misura la differenza diretta nelle probabilità.

$$\text{PredDiff}(f_i(x)) = f(x) - f(x \setminus A_i) = p(y = c|x) - p(y = c|x \setminus A_i) \quad (49)$$

Dove $p(y = c|x)$ è la probabilità che il modello assegna alla classe c avendo tutte le informazioni, mentre $x \setminus A_i$ rappresenta l'istanza x senza l'informazione della feature A_i .

2. InfoDiff

- Misura la differenza in termini di contenuto informativo (utilizzando il logaritmo in base 2).

$$\text{InfoDiff}(f_i(x)) = \log_2 p(y = c|x) - \log_2 p(y = c|x \setminus A_i) \quad (50)$$

8.6.3. Come si rimuove la feature?

In un tipico modello di machine learning addestrato su N variabili, non possiamo fisicamente passare un input con $N - 1$ variabili, altrimenti il sw ci darebbe errore, questo anche perché non possiamo semplicemente togliere il valore dato che la funzione f cambierebbe.

Per risolvere questo problema, PredDiff simula la rimozione della feature marginalizzandola, ovvero calcolando un «valore medio» di previsione:

$$p(y|x \setminus A_i) = \sum_{s=1}^n p(A_i = a_s) \cdot p(y|x \leftarrow A_i = a_s) \quad (51)$$

Il sistema prende l'istanza x e sostituisce il valore della feature A_i con **tutti i possibili valori a_s** che quella feature può assumere nel dataset. Per ognuno di questi valori calcola la previsione del modello e la moltiplica per la probabilità $p(A_i = a_s)$ di osservare quel valore nella realtà. Sommando il tutto, ottiene una previsione «media» che isola (e quindi annulla) l'effetto specifico che A_i aveva sull'istanza x .

8.6.4. Limiti e criticità

Questo tipo di approccio ha 3 limiti principali:

1. **Model-Agnostic (😞):** Tratta il modello come una scatola nera, il che lo rende flessibile ma computazionalmente molto pesante, poiché richiede di interrogare il modello innumerevoli volte per calcolare tutte le probabilità marginali.
2. **Crea dati irrealistici (😞):** Perturbare le feature combinandole artificialmente può creare dati irrealistici. Questo è il noto problema chiamato ODD o Out-of-Distribution. Sostituendo valori in modo forzato per calcolare la media, potremmo creare combinazioni assurde (es. incrociare «Sesso=uomo» con «Gravidanza=vero»). Interrogando il modello su dati irrealistici, che non ha mai visto in fase di addestramento, il modello restituirà previsioni arbitrarie e inaffidabili, falsando il risultato.
3. **Interazione tra Feature (😞):** PredDiff valuta l'impatto di una variabile alla volta. Questo è un grosso problema per modelli non lineari complessi, in cui le variabili spesso interagiscono tra di loro. Calcolare l'importance singola non cattura le interazioni multiple.

Proprio per superare queste limitazioni è stato introdotto **SHAP**. Per comprenderlo peò bisogna prima introdurre la teoria dei giochi.

8.7. Game theory

La Teoria dei Giochi analizza le dinamiche decisionali definendo tre elementi fondamentali, ovvero: **I Giocatori**, **le Strategie** e **il Payoff**

8.7.1. Il Dilemma del Prigioniero e l'Equilibrio di Nash

L'esempio classico riportato per spiegare le strategie non cooperative è il Dilemma del Prigioniero, in cui due giocatori (P_1 e P_2) hanno due possibili strategie: confessare o non confessare. La matrice dei payoff mostra i seguenti «costi» (es. anni di prigione):

- Se entrambi confessano, ottengono un payoff di (6, 6).
- Se entrambi non confessano, ottengono la «strategia migliore» globale con un payoff di (1, 1), ma questo è possibile solo se cooperano.
- Se uno confessa e l'altro no, chi confessa ottiene 0 e chi non confessa ottiene 10.

In questo scenario emerge il concetto di **Equilibrio di Nash**, definito come quella situazione in cui «nessun giocatore può migliorare il proprio payoff cambiando strategia, a parità di condizioni (other things being equal)».

Nel dilemma, la situazione (1, 1) non è un equilibrio, perché ogni giocatore razionale capirà che, tradendo l'altro e passando alla strategia «confessa», può migliorare il proprio risultato personale. Di conseguenza, l'unico vero Equilibrio di Nash verso cui i giocatori convergeranno (non fidandosi l'uno dell'altro) è **(6, 6)**, ovvero confessare entrambi.

Viene illustrato anche un altro tipo di equilibrio tramite il gioco Sasso-Carta-Forbice.

Poiché non esiste una singola strategia vincente a priori, l'Equilibrio di Nash si raggiunge tramite una strategia mista, in cui ogni opzione viene giocata con una probabilità di 1/3, introducendo il concetto matematico di «tirare a caso». Questo principio è formalizzato dal **Teorema di Nash**, il quale stabilisce che ogni gioco con informazione completa e n giocatori possiede sempre un equilibrio di Nash, eventualmente proprio attraverso strategie miste.

8.7.2. I valori di Shapley per le coalizioni

Passando dai giochi competitivi a quelli cooperativi, si pone un nuovo problema:

come allocare correttamente e giustamente il «credito» (o il profitto) tra vari giocatori che hanno formato una coalizione, considerando che ognuno apporta un contributo diverso?

Per risolvere questo problema, **Lloyd Shapley** nel 1953 (Premio Nobel nel 2012) formulò i **Valori di Shapley**, che assegnano il credito in base a quattro **Assiomi di Equità** (Fairness Axioms):

- **Efficienza (Efficiency)**: La somma dei crediti distribuiti a tutti i giocatori deve essere uguale al valore totale generato dall'intera coalizione (*grand coalition*).
- **Simmetria (Symmetry)**: Se due giocatori sono intercambiabili e portano lo stesso identico valore alla coalizione, riceveranno lo stesso credito.
- **Giocatore Nullo (Null Player)**: Se un giocatore non aggiunge alcun valore alla coalizione, il suo valore di Shapley sarà 0.

- **Linearità (Linearity):** I crediti calcolati per combinazioni lineari di giochi si comportano in modo lineare.

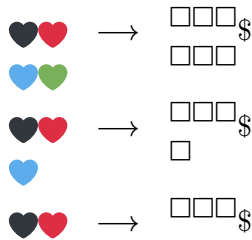
L'algoritmo calcola matematicamente l'importanza di un giocatore valutando il suo contributo in base a «quanti modi si hanno per generare una coalizione» e calcolando l'apporto marginale che egli aggiunge a ciascun sottoinsieme possibile.

8.8. SHAP (SHapley Additive exPlanations)

È un approccio unificato per interpretare le predizioni dei modelli.

Dato un insieme di giocatori $D = \{1, \dots, d\}$, un **gioco** è dato specificando per ogni **coalizione** un valore che rappresenta il **profitto** per quella coalizione: $n : 2^D \rightarrow \mathbb{R}$ **profitto** o **funzione caratteristica**.

Esempio: Dipendenti in una azienda



Sulla sinistra sono rappresentati i dipendenti (ovvero i giocatori) mentre i quadratini sulla destra rappresentano il guadagno che ogni possibile coalizione di dipendenti porta all'azienda. Il problema è capire come allocare il **credito** tra i giocatori considerando che ogni dipendente apporta un diverso contributo (anche a seconda della coalizione). Le funzioni che risolvono questo problema sono le **Shapley values**, derivate da un insieme di *assiomi di fairness*. L'idea è sfruttare questo concetto per dare un'importanza alle feature.

8.8.1. Shapley values

Sia G l'insieme dei giochi su d giocatori. Le Shapley values assegnano un vettore di crediti a ogni gioco (in $\mathbb{R}^d$, un credito per ogni giocatore): $\phi : G \rightarrow \mathbb{R}^d$. Per un gioco v , le Shapley values sono

$$\underbrace{\phi_1(v), \phi_2(v), \dots, \phi_d(v)}_{\text{come dovremmo assegnare crediti basandoci su assiomi di fairness}}$$

Assiomi di fairness (Fairness Axioms)

1. Efficiency

La somma dei crediti deve essere pari al valore della **ground coalition** (coalizione con tutti i giocatori) meno il valore della **null coalition** (coalizione con nessun giocatore):

$$\sum_{i \in D} \phi_i(v) = \underbrace{v(D)}_{\text{GRAND COALITION}} - \underbrace{v(\emptyset)}_{\text{NULL COALITION}} \quad (52)$$

Solitamente, il valore della **null coalition** è 0.

2. Symmetry

Se due giocatori i, j sono interscambiabili (ovvero $v(S \cup \{i\}) = v(S \cup \{j\}) \forall S \subseteq D, i, j \notin S$, quindi il loro apporto è uguale), allora $\phi_i(v) = \phi_j(v)$.

3. Null player

Se un giocatore non apporta contributo (contribuisce con valore nullo), ovvero $v(S \cup \{i\}) = v(S) \forall S \subseteq D, i \notin S$, allora $\phi_i(v) = 0$.

4. Linearity

I crediti si comportano linearmente rispetto a combinazioni lineari dei giochi, ovvero
 $\phi(c_1 v_1 + c_2 v_2) = c_1 \phi(v_1) + c_2 \phi(v_2)$ con $c_1, c_2 \in \mathbb{R}$.

Le Shapley values sono le uniche funzioni $\phi : G \rightarrow \mathbb{R}^d$ che soddisfano gli assiomi:

$$\begin{aligned} \phi_j &= \frac{1}{d} \sum_{\substack{\text{Fissata la} \\ \text{colazione} \\ \downarrow \\ \text{Media pesata su} \\ \text{tutte le coalizioni} \\ S \subseteq D \setminus j}} \frac{1}{\binom{d-1}{|S|}} (v(S \cup \{j\}) - v(S)) = \\ &= \frac{1}{d} \sum_{S \subseteq D \setminus j} \frac{|S|!(d-1-|S|)!}{(d-1)!} (v(S \cup \{j\}) - v(S)) = \\ &= \sum_{S \subseteq D \setminus j} \frac{|S|!(d-1-|S|)!}{d!} (v(S \cup \{j\}) - v(S)) \end{aligned} \quad (53)$$

Choose $|S|$ players from $d-1$.
 Quanti modi ho di generare una coalizione grande $|S|$
 a partire da $d-1$ (si fa il reciproco per avere la media)

Esempio: Gioco con tre giocatori, $D = \{1, 2, 3\}$

Come si calcolano le Shapley values per il giocatore 2?

$$S_1 = \emptyset \rightarrow \frac{0!2!}{3!} (v(\{2\}) - v(\emptyset)) = \frac{1}{3} (v(\{2\}) - v(\emptyset)) \quad (54)$$

$$S_2 = \{1\} \rightarrow \frac{1!1!}{3!} (v(\{1, 2\}) - v(\{1\})) = \frac{1}{6} (v(\{1, 2\}) - v(\{1\})) \quad (55)$$

$$S_3 = \{3\} \rightarrow \frac{1!1!}{3!} (v(\{3, 2\}) - v(\{3\})) = \frac{1}{6} (v(\{3, 2\}) - v(\{3\})) \quad (56)$$

$$S_4 = \{1, 3\} \rightarrow \frac{2!0!}{3!} (v(\{1, 3, 2\}) - v(\{1, 3\})) = \frac{1}{3} (v(\{1, 3, 2\}) - v(\{1, 3\})) \quad (57)$$

8.8.2. Applicazione di SHAP a AI/ML

- Players $\rightarrow$ Features
- Profit $\rightarrow$ Comportamento del modello (accuracy/loss)
- Shapley values $\rightarrow$ Quantificano l'importanza delle feature

Il metodo **Shap** utilizza le Shapley values per spiegare **singole predizioni** (quindi stiamo parlando di **spiegazioni locali**):

Additive Feature Distribution

$$f(\hat{x}) = \phi_0 + \sum_{j=1}^d \phi_j \hat{x}_j \quad (58)$$

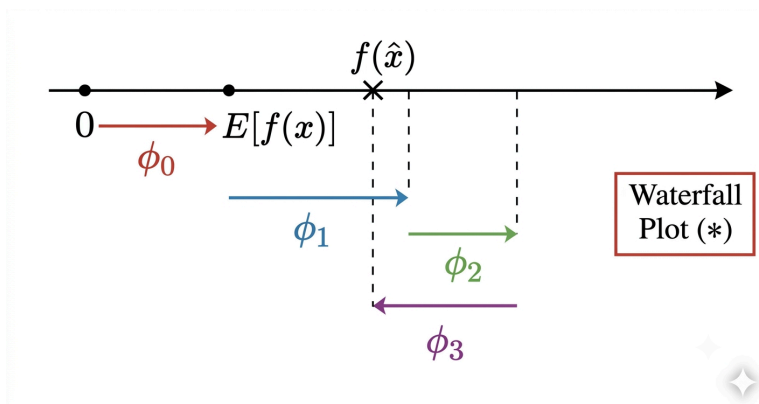
Istanza di test $\rightarrow$ ϕ_0 $\leftarrow$ $E[f(x)]$ $\leftarrow$ $\phi_j \hat{x}_j$
valore atteso di $f(x)$
ovvero la predizione «media»

Shapley values $\leftarrow$ ϕ_j $\leftarrow$ $\hat{x}_j$
Rappresentazione interpretabile: vettore di coalizioni

Quindi «misura» quanto cambia la predizione di test rispetto alla predizione media:

$$\begin{aligned} &\phi_0 + \sum_{j=1}^d \phi_j \hat{x}_j \\ &\text{Predizione media} \quad \text{spostamento} \quad \text{quanto impattano le features in funzione dei loro Shapley values} \end{aligned} \quad (59)$$

Per rappresentare graficamente possiamo usare un **Waterfall plot**:



Feature 7: $\rightarrow$

Feature 2: $\rightarrow$

Feature 4: $\leftarrow$

Le frecce **rosse** indicano che la feature spinge verso la classe positiva mentre quelle **blu** indicano che la feature spinge verso la classe negativa. La lunghezza delle frecce indica il contributo della feature.

8.8.3. Problema computazionale

Per calcolare le Shapley values esatte, avremmo bisogno di sommare tutte le possibili coalizioni (ovvero sotto-insieme di feature) e ciò diventa intrattabile per $d > 20$ dato che il numero di sotto-insiemi è esponenziale rispetto a d .

8.8.3.1. Tecniche per approssimare gli Shapley values

Alcuni sono **model agnostic** (come **KernelSHAP**) mentre altri sono **specifici per il modello** (per esempio **TreeSHAP** per le foreste).

8.8.4. Come possiamo ottenere statistiche globali a partire dalle spiegazioni locali date dalle Shapley values?

Si calcola la media del valore assoluto delle Shapley value dei dati:

$$I_j = \sum_{j=1}^d |\phi_j^{(n)}| \quad (60)$$

↑ Feature importance
 ↖ A causa del valore assoluto, si perdono le informazioni riguardo la direzione dell' importance

8.9. KernelSHAP ($\approx$ LIME)

Tecnica per stimare il contributo di una feature per una predizione $\hat{x}$ come un'approssimazione delle Shapley values. Nasce dalla necessità di dover approssimare i valori della black-box (analogamente a quanto succede con LIME che campiona). É agnostico rispetto al modello e fornisce spiegazioni **locali**.

Idea per i modelli localmente interpretabili:

- In **LIME** si ha

$$\text{explanation}(x) = \underset{g \in G}{\operatorname{argmin}} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (61)$$

- In **Kernel shap**:

- Si imposta $\Omega(g) = 0$

- Si imposta $\mathcal{L}(f, g, \pi_x) = \sum_{z' \in Z} \pi_x(z') (f(x) - g(z'))^2$ dove

$$\pi_x(z') = \frac{d-1}{\binom{d}{|z'|} |z'| (d - |z'|)} \quad (62)$$

e $|z'|$ è il numero di elementi non-zero in z'

- Avendo questa impostazione, KernelSHAP $\approx$ LIME.

8.9.1. Schema dell'algoritmo

1. Campiona le coalizioni

$$z'_k \in \{0, 1\}^d, k \in \{1, \dots, N\} \quad (63)$$

dove N è il numero di campioni. Sarà quindi un vettore del tipo

$$[0 \ 1 \ 1 \ 0 \ 0 \ 0 \ 1] \quad (64)$$

dove l'1 alla posizione i indica che la i -esima feature è presente nel k -esimo campione.

2. Calcola la f per i campioni (ovvero il valore della black-box che vogliamo spiegare) e applica una maschera

$$h_x(z'_k) : \{0, 1\}^d \rightarrow \mathbb{R}^p \quad (65)$$

dove z'_k è il campione e p è il numero di feature nel dominio originale di f .

Esempio con dati tabulari (usando il vettore di esempio sopra):

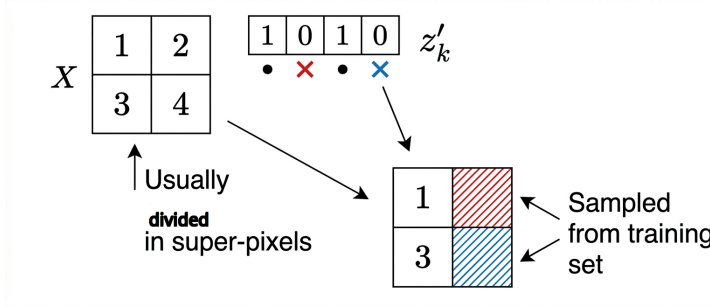
Partendo da

$$x[0.7 \ 1 \ 0.5 \ -2.9 \ 9 \ 7 \ \text{BLUE}] \quad (66)$$

Essendo dati tabulari possono mischiare valori numerici con valori categorici. A questo punto viene applicata la maschera seguendo il vettore di (0,1): le feature nelle posizioni corrispondenti agli 1 vengono prese come sono mentre le altre vengono sostituite con valori campionati casualmente dal training set. Nel caso dell'esempio quindi avremo

$$[x \ 1 \ 0.5 \ x \ x \ x \ \text{BLUE}] \quad (67)$$

Esempio con immagini:



3. Calcola il peso π della coalizione:

$$\pi_x(z') = \frac{d-1}{\binom{d}{|z'|} |z'| (d-|z'|)} \quad (68)$$

↑
numero di elementi non nulli in z

4. Fitta modello lineare pesato:

$$\mathcal{L}(f, g, \pi_x) = \sum_{z' \in Z} \pi_x(z') (f(x) - g(z'))^2, \Omega(g) = 0 \quad (69)$$

↑
a differenza di Lime, non viene fissato
un numero massimo di feature per spiegazione

5. Restituisce i coefficienti del modello imparato:

I coefficienti restituiti rappresentano una approssimazione delle Shapley Values: l'i-esimo componente del modello lineare equivale alla Shapley Value ϕ_i .

8.9.2. LIME e KernelSHAP

Entrambi arrivano ad usare le Shapley Values ma partendo da concetti diversi:

- Lime si basa su 4 assiomi di fairness e le Shapley Values sono le uniche funzioni che le rispettano
- KernelSHAP si basa su 3 proprietà di attribuzione delle feature e, anche in questo caso, le Shapley Value sono le uniche funzioni che le rispettano. Nel paragrafo successivo introduciamo queste 3 proprietà.

8.9.2.1. Proprietà di attribuzione delle feature

1. Accuratezza locale

g deve approssimare f localmente:

$$f(x) = g(z') \quad (70)$$

↑
 $\phi_0 + \sum_{i=1}^d \phi_i z'_i$

2. Missingness

$z'_i = 0 \rightarrow \phi_i = 0 \sim$ player nullo

3. Consistenza

Se ho due modelli (diversi) f_1 e f_2 (black boxes) e il contributo di una feature j in f_1 è maggiore rispetto che in f_2 allora la Shapley value in f_1 deve essere maggiore rispetto alla corrispondente in f_2 ovvero $\phi_j(f_1, x) \geq \phi_j(f_2, x)$, infatti $f_1(x) - f_2\left(\frac{x}{j}\right) \geq f_2(x) - f_1\left(\frac{x}{j}\right) \rightarrow \phi_j(f_1, x) \geq \phi_j(f_2, x)$.

Le Shapley values sono le uniche funzioni che rispettano tutte e 3 le proprietà, le quali invece non sono soddisfatte da LIME dato che pone un vincolo sul numero di feature ($\Omega(g) \neq 0$).

8.10. Metodi basati sul gradiente e Mappe di salienza

L'idea è sfruttare le informazioni sul gradiente date le feature in input per capire quali sono le più importanti per la decisione: tipicamente vengono usati per le reti neurali e esistono diversi approcci a seconda di come viene calcolato il gradiente.

In questi metodi, la spiegazione ha la stessa dimensione dell'input perché ad ogni parte dell'input viene assegnato un valore che rappresenta la sua **importanza/salienza** (per esempio, ogni pixel di un'immagine o ogni parola di un testo).

La base del metodo consiste nel calcolare il gradiente considerando le feature in input per poi analizzare come i cambiamenti all'input influenzano l'output. Per farlo calcoliamo il valore nel gradiente in x_0 e questo rappresenterà l'importance della feature x_o

$$\left. \frac{\partial F_c}{\partial x} \right|_{x_0} \quad (71)$$

Possiamo approssimare il valore di F_c con un'approssimazione di Taylor del primo ordine (**Vanilla gradient**):

$$F_c(x) \approx w^t x + b \quad (72)$$

Un valore maggiore del gradiente indica che piccoli cambiamenti alle feature corrispondenti comportano un più significativo cambiamento nell'output del modello.

8.10.1. Mappe di salienza

Funzione che associa un immagine a colori ad un insieme di feature:

$$I_0 \in \mathbb{R}^{H \times W \times K} \rightarrow M \in \mathbb{R}^{H \times W}$$

$$M_{ij} = \max_k |w_{ijk}|$$

con $k = 3$ per immagini RGB.

8.10.1.1. Smooth-grad

Le mappe ottenute col Vanilla gradient risultano molto *rumorose*. Per rimediare, possiamo fare una media per ottenere mappe più robuste: $W = \frac{1}{N} \sum_{i=1}^N \nabla_x F_c(x + \varepsilon)$ ottenendo un effetto di smoothing.

↑
copie rumorose dell'immagine in input

8.10.1.2. Grad-cam

CAM: **Class Activation Mapping**.

Invece di calcolare il gradiente sulle feature di input, viene calcolato sull'output dell'ultimo layer convoluzionale:

- k feature maps nell'ultimo layer convoluzionale:
- Grad-cam calcola

$$\frac{\partial F_c}{\partial A_{ij}^h} \quad (73)$$

Mappa di salienza della classe c calcolata da Gradcam:

$$S_{\text{gradcam}}^c = \text{ReLU} \left(\sum_h \alpha_h^c A^h \right) \quad (74)$$

$\uparrow$
 calcolato da GRAD-CAM

8.10.1.2.1. Algoritmo

1. Passaggio *forward* della rete
2. Si prende lo score della classe prima del softmax y^c
3. Imposta le score delle altre classi a 0
4. Fai backpropagation fino all'ultimo layer convoluzionale: $\frac{\partial y^c}{\partial A_{ij}^h}$
5. Si calcola l'importanza media della feature map A^h per la classe c:

$$\alpha_h^c = \frac{1}{Z} \sum_{u,v} \left(\frac{\partial y^c}{\partial A_{u,v}^h} \right) \quad (75)$$

6. Calcola

$$S = \text{ReLU} \left(\sum_h \alpha_h^c A^h \right) \quad (76)$$

che non avrà la stessa dimensione dell'immagine in input (è più *fine* rispetto a **Vanilla grad**).

8.10.1.3. Guided Gradcam

L'idea è combinare le spiegazioni di Gradcam con quelle ottenute a livello di pixel (pre esempio da Vanilla grad):

$$S_{\text{GGC}} = \text{vsample}(S_{\text{GC}}) \odot S_{\text{VG}} \quad (77)$$

$\uparrow$ Saliency guided Gradcam $\uparrow$ Saliency Gradcam $\uparrow$ Prodotto pointwise $\leftarrow$ Saliency Vanilla Gradient

8.10.2. Sanity checks per Mappe di salienza

In questo paper vengono proposti due test sperimentali per valutare la bontà delle mappe di salienza:

1. Confronta le mappe di salienza di una rete addestrata su un certo dataset con quelle di una rete inizializzata randomicamente $\rightarrow$ se non ci sono differenze, le mappe di salienza non possono essere una buona spiegazione per quel modello.
2. Confronta le mappe di salienza di una rete addestrata su un certo dataset con quelle di una rete addestrata sulle stesse immagini di input ma con le label perturbate $\rightarrow$ se non ci sono differenze, le mappe di salienza non possono essere una buona spiegazione per quel dataset.

Vanilla, SmoothGrad e GRADCAM passano il test, mentre Guided GRADCAM no.

8.11. Modelli con Attention

Il layer di **attention** in una rete neurale confronta parti dell'input (parole di una frase o patch di un immagine) con l'obiettivo di costruire rappresentazioni (es. per la

classificazione).

Potrebbe questo meccanismo fornire una sorta di «spiegazione intrinseca»?

- Es.

The room service was excellent positiva

α_1 α_2 α_3 α_4 α_5

Se si guarda i pesi di attenzione α_i per fare questa sentiment analysis, α_3 e α_5 hanno una maggiore influenza

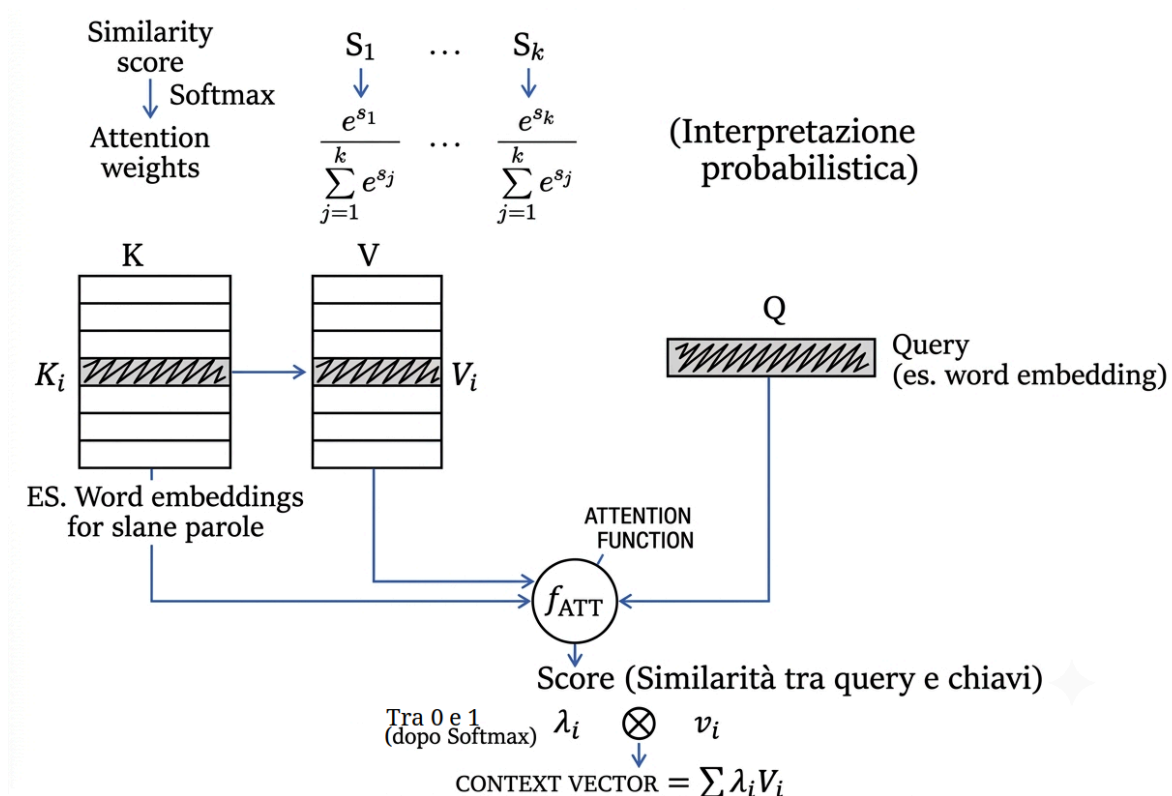
				α_1	α_2	α_3	α_4
				α_5	α_6	α_7	α_8
				α_9	α_{10}	α_{11}	α_{12}
				α_{13}	α_{14}	α_{15}	α_{16}

-

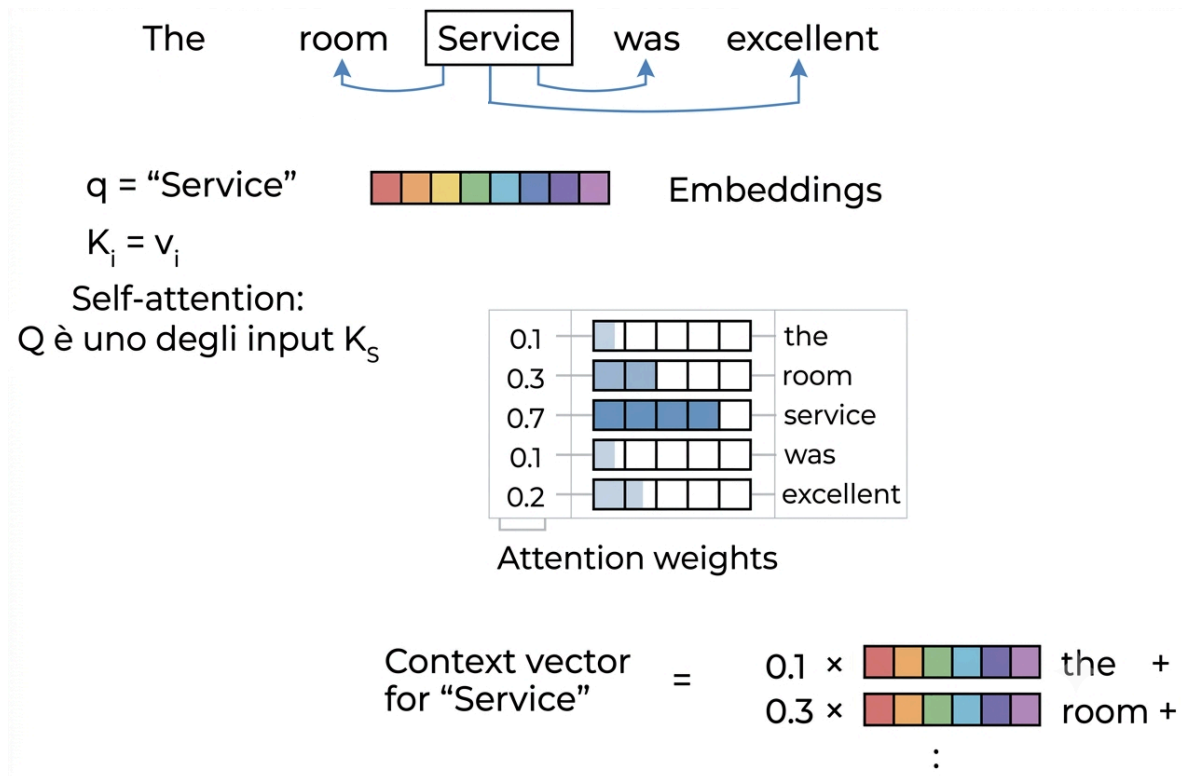
Pesi di attenzione per ogni porzione dell'immagine in input

Si potrebbe quindi pensare ad un processo selettivo per trovare le parti più rilevanti e imparare ad assegnare un peso ad ogni parte.

Questo si può fare con delle **query**, quindi, mappando una query Q e delle chiavi K per calcolare gli *Score Scalari* (similarity), questi poi vengono dati ad una softmax per ottenere un insieme di pesi di attenzione.



Esempio:



Ci sono molte alternative per calcolare la similarità tra queste *query* e *chiavi*:

- score additivo

$$\alpha = \text{softmax} \left(\underset{\substack{\uparrow \\ \text{trainable}}}{W_3^T} \cdot \tanh \left(\underset{\substack{\uparrow \\ \text{parametri}}}{W_1 K} + W_2 Q \right) \right) \quad (78)$$

- prodotto scalare

$$\alpha = \text{softmax} \left(\frac{KQ}{\sqrt{m}} \right) \quad (79)$$

- in generale

$$\alpha_i = \frac{\exp(f_{\text{att}}(k_i, q))}{\sum_j \exp(f_{\text{att}}(k_j, q))} \quad (80)$$

L'attenzione è una spiegazione?

Possiamo interpretare i pesi di attenzione come un elemento di spiegazione?

[Jain and wallace 2019, Attention is not explanation]

[Wiegrefe and Pinter 2019, Attention is not not explanation]

- Jain and Wallace: Propongono due esperimenti
 - 1 - Misurano la correlazione tra attention weights e altre (es. gradient based) tecniche per feature importance e gli dà dei risultati non buoni, quindi non correlano molto l'attention weights e gli altri metodi.
 - 2 - shuffle random dei pesi di attenzione e osservano i cambiamenti nell'output (anche adversarial shuffling).

Anche qui non si vedono grandi cambiamenti nell'output.

- Wiegrefe and Pinter:

Obiezione 1: la correlazione, tra attention e altre tecniche, non per forza è una buona metrica. In particolare la correlazione può essere una metrica debole per altre tecniche di feature importance.

Obiezione 2: Gli esperimenti di Jain and Wallace considerano solo Faithful explanation (e non quelle plausibili). I pesi dell'attention possono dare una spiegazione plausibile.

8.12. Ceteris Paribus Plots

Versione in latino di «Al pari di», modello agnostico per *outcome explanation/model inspection*.

L'idea è costruire un grafico per ogni feature dato un singolo esempio di test:

- Si **perturba** il valore di quella feature
- Si prova una **griglia** di possibili valori
- Si ridà in pasto alla black box l'esempio con i valori modificati
- Si **disegna** il grafico

Ci sono due possibilità:

1. La feature è continua:

- Sia $\hat{x}$ l'esempio di test e j la feature
- Creiamo una griglia di valori $z_1, \dots, z_k$ dove, solitamente, $z_1 = \min_{x \in \mathcal{X}}(x_j)$ e $z_k = \max_{x \in \mathcal{X}}(x_j)$
- Per ogni z_i :
 - Creiamo un nuovo data point $\tilde{x} = \hat{x}_{\hat{x}_j = z_i}$ (ovvero nella posizione j sostituisco il valore con z_i)
 - Calcoliamo $f(\tilde{x})$
 - Plottiamo il punto $(z_i, f(\tilde{x}))$
- Plottiamo anche il punto originale

Esempio:

2. La feature è categorica:

- Sia $\hat{x}$ l'esempio di test e j la feature
- Si considerano tutti i possibili valori di j : $z_1, \dots, z_k$
- Per ogni z_i :
 - Creiamo un nuovo data point $\tilde{x} = \hat{x}_{\hat{x}_j = z_i}$
 - Calcoliamo $f(\tilde{x})$
 - Disegno un bar plot $(z_i, f(\tilde{x}))$
- Plottiamo anche la barra relativa al punto originale

Esempio:

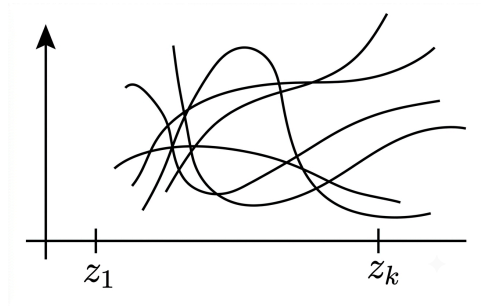
Vantaggi:

- È un modello agnostico
- È abbastanza semplice e intuitivo

Svantaggio: c'è la possibilità di creare dati non realistici o comunque fuori dalla distribuzione perché non vengono considerate le relazioni tra le variabili.

8.12.1. Individual conditional expectation (ICE)

Si disegna in un singolo grafico tutti i **Ceteris paribus** per una data feature j (quindi un grafico per esempio, tutti sulla stessa feature).



Per migliorare la leggibilità, possiamo disegnare l'ICE centrato: centriamo le curve rispetto ad un certo punto (per esempio z_1) e disegniamo le differenze/offset (per esempio possiamo ancorare al minimo): $C-ICE(x_j) = ICE(x_j) - f(\hat{x}_{x_j=z_1})$

8.12.2. Partial dependence plots

Media degli ICE plot su tutto il dataset.

x_s = feature di interesse

x_c = tutte le altre feature (il complemento)

$$PD_{x_s} = E_{x_c}[f(x_s, x_c)] = \int f(x_s, x_c) dP(x_c) \approx \frac{1}{n} \sum_{i=1}^n f(x_s, x_c^{(i)})$$

$\uparrow$ distribuzione del dato $\uparrow$ campionato sul test set
 (volendo potrebbe essere fatto anche sul training set)

Può essere fatto anche nella formulazione 2D, ovvero considerando due feature: in questo caso il grafico diventa una heatmap: GRAFICO.

8.13. Permutation feature importance

É un'altra tecnica per la *feature attribution* (quindi, per esempio, per capire l'importanza di una feature), nella classe dei *model inspection* e *model agnostic*.

L'idea é questa: immagino il dato tabulare come una colonna e permuto i valori per quella feature (scambio i valori nella colonna): IMMAGINE.

Mi aspetto che scambiando questi valori e ricalcolando lo score (per esempio accuracy o F1 o altro) riesca a quantificare quanto conta la feature.

Quindi, calcolo lo score del classificatore prima e dopo la permutazione: se non c'è un cambiamento significativo, l'importanza della feature é bassa:

$$S_{\text{ORIG}} = \frac{1}{n_{\text{TEST}}} \sum_{i=1}^{n_{\text{TEST}}} s(y_i, f(x_i)) \quad (81)$$

$\uparrow$ score originale $\uparrow$ numero di esempi nel test set

$$S_{\text{PERM}(j)} = \frac{1}{n_{\text{TEST}}} \sum_{i=1}^{n_{\text{TEST}}} s(y_i, f(x_i^{p_j})) \quad (82)$$

$\uparrow$ score avendo permutato la feature j $\uparrow$ esempio x_i con permutazione della feature j

$$\text{FI}(j) = S_{\text{ORIG}} - S_{\text{PERM}(j)} \text{ oppure } \frac{S_{\text{ORIG}}}{S_{\text{PERM}(j)}} \quad (83)$$

$\uparrow$
 feature importance di j

La procedura può essere ripetuta per diverse permutazioni (aumentando la robustezza).